模型机设计报告

班级 计科 210X 姓名 甘晴void 学号 202108010XXX

一、设计目的

完整、连贯地运用《电路与电子学》所学到的电路、模电和数电知识，熟练掌握现代 EDA 工具基本使用方法，为后续课程学习和今后从事相关工作打下良好的基础或做下一些铺垫。

二、设计内容

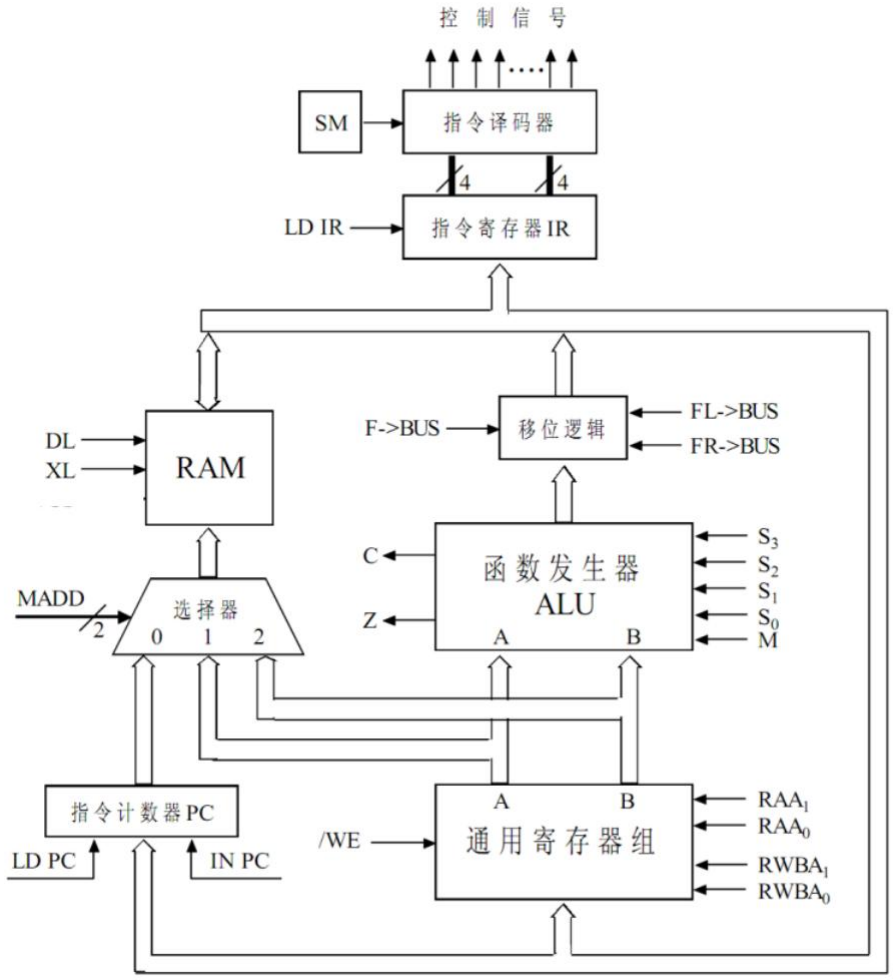
① 设计一个由传感器（电桥）、放大器、滤波器、模数转换器构成的模拟通道；

② 按照给定的数据通路、数据格式和指令系统，使用 EDA 工具设计一台用硬连线逻辑控制的简易计算机；

③ 用所设计的计算机控制模拟通道对外界模拟输入信号进行采样、离散和处理，并显示在数码显示管上；

三、详细设计

3.1 设计的整体架构



3.2 各模块的具体实现

（此部分必须有模块的接口设计，功能实现，功能的仿真验证等内容。）

3.2.1 指令译码器

（1）接口设计:

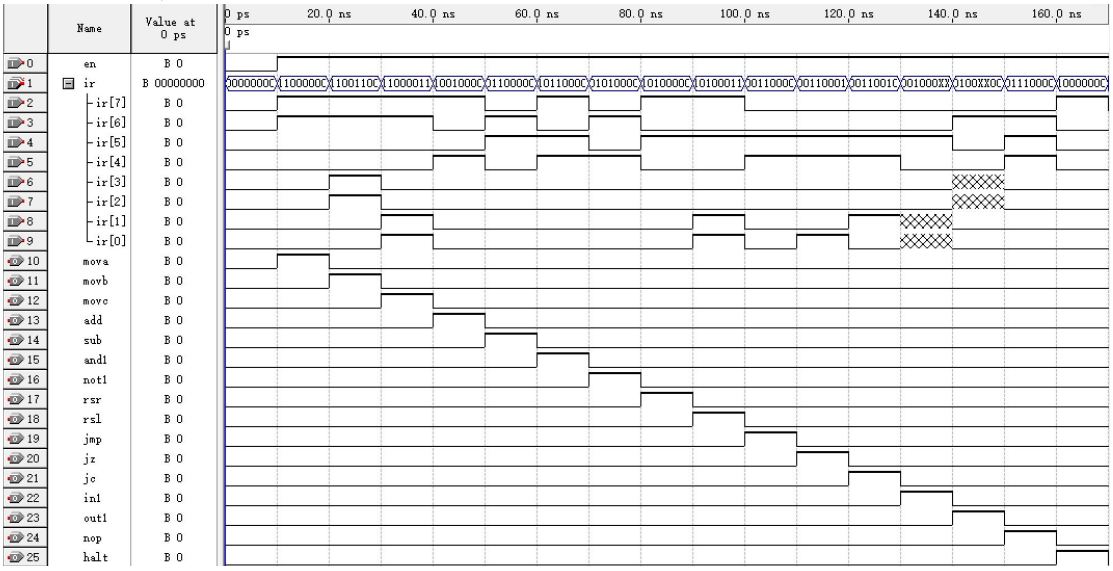
```
input [7:0] ir;
input en;
```

output mova,movb,movc,add,sub,and1,not1,rsr,rsl,jmp,jz,jc,in1,out1,nop,halt;

（2）功能实现:

判定是哪条指令，则对应指令的输出为 1，否则输出为 0。

（3）功能仿真验证:



结果分析及结论:

分析：功能仿真是指不考虑器件延时和布线延时的理想情况下对源代码进行逻辑功能的验证。由仿真波形可得，对于输入状态的变化，输出结果实时变化，没有延迟，其结果与电路设计的真值表的结果相对应。

当 en 为 0 时，不管 ir 为何值，16 个输出全为 0

当 en 为 1 时：

当 ir=11000000 时，mova 输出为 1；

当 ir=11001100 时，movb 输出为 1；

当 ir=11000011 时，movc 输出为 1；

当 ir=10010000 时，add 输出为 1；

当 ir=01100000 时，sub 输出为 1；

当 ir=10110000 时，and1 输出为 1；

当 ir=01010000 时，not1 输出为 1；

当 ir=10100000 时，rsr 输出为 1；

当 ir=10100011 时，rsl 输出为 1；

当 ir=00110000 时，jmp 输出为 1；

当 ir=00110001 时，jz 输出为 1；

当 ir=00110010 时，jc 输出为 1；

当 ir=00100000 时，in1 输出为 1；

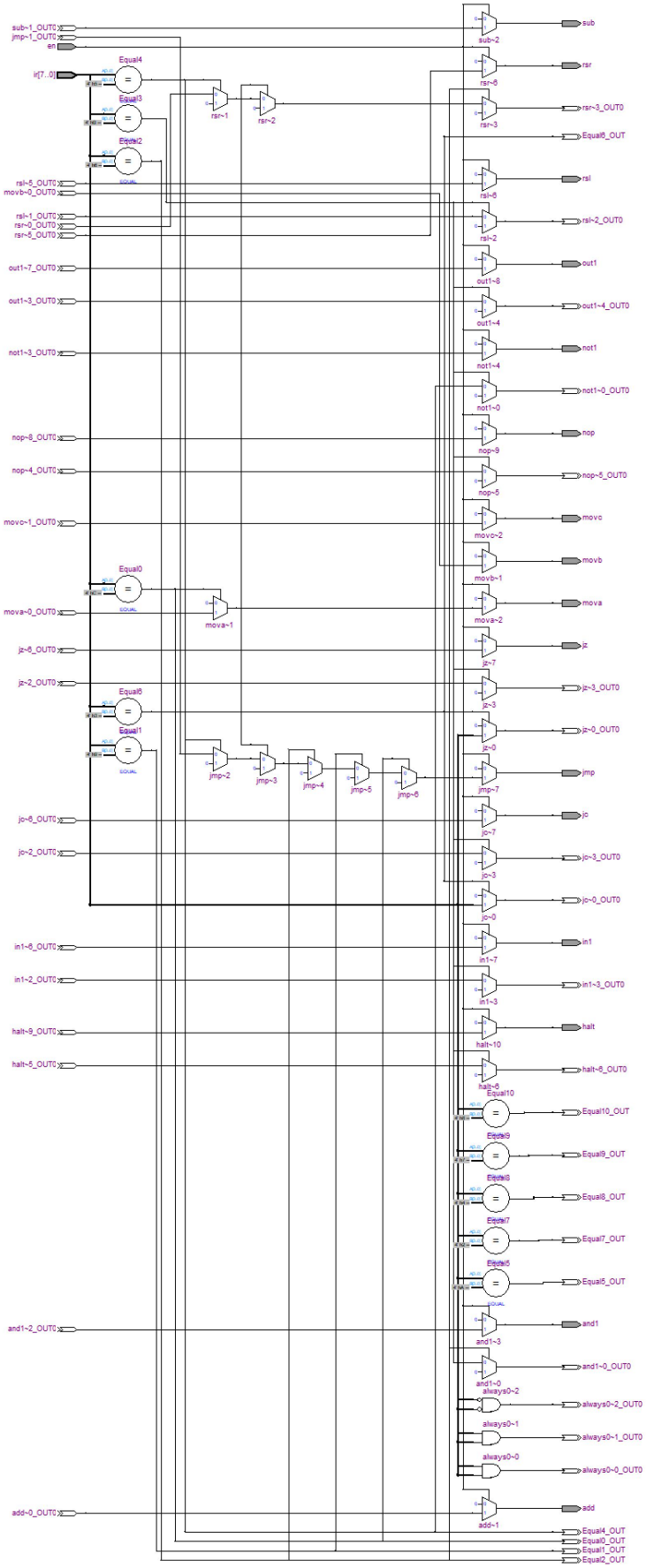
当 ir=01000000 时，out1 输出为 1；

当 ir=01110000 时，nop 输出为 1；

当 ir=10000000 时，halt 输出为 1；

结论：功能仿真操作简单，能体现和验证实验的功能，但忽略延迟的影响会使结果与实际结果有一定误差。

（4）RTL 视图


sub~1_OUT0
jmp~1_OUT0
en
ir[7..0]
Equal4
Equal3
Equal2
rsl~5_OUT0
movb~0_OUT0
rsl~1_OUT0
rsr~0_OUT0
rsr~5_OUT0
out1~7_OUT0
out1~3_OUT0
not1~3_OUT0
nop~8_OUT0
nop~4_OUT0
movc~1_OUT0
Equal0
mova~0_OUT0
jz~6_OUT0
jz~2_OUT0
Equal6
Equal1
jc~6_OUT0
jc~2_OUT0
in1~6_OUT0
in1~2_OUT0
halt~9_OUT0
halt~5_OUT0
and1~2_OUT0
add~0_OUT0
rsr~1
rsr~2
mova~1
jmp~2
jmp~3
jmp~4
jmp~5
jmp~6
sub
sub~2
rsr
rsr~6
rsr~3
rsr~3_OUT0
Equal6_OUT
rsl
rsl~6
rsl~2_OUT0
rsl~2
out1
out1~8
out1~4_OUT0
out1~4
not1
not1~4
not1~0_OUT0
not1~0
nop
nop~9
nop~5_OUT0
nop~5
movc
movc~2
movb
movb~1
mova
mova~2
jz
jz~7
jz~3_OUT0
jz~3
jz~0_OUT0
jz~0
jmp
jmp~7
jc
jc~7
jc~3_OUT0
jc~3
jc~0_OUT0
jc~0
in1
in1~7
in1~3_OUT0
in1~3
halt
halt~10
halt~6_OUT0
halt~6
Equal10
Equal10_OUT
Equal9
Equal9_OUT
Equal8
Equal8_OUT
Equal7
Equal7_OUT
Equal5
Equal5_OUT
and1
and1~3
and1~0_OUT0
and1~0
always0~2
always0~2_OUT0
always0~1
always0~1_OUT0
always0~0
always0~0_OUT0
add
add~1
Equal4_OUT
Equal0_OUT
Equal1_OUT
Equal2_OUT

视图分析及结论:

分析：

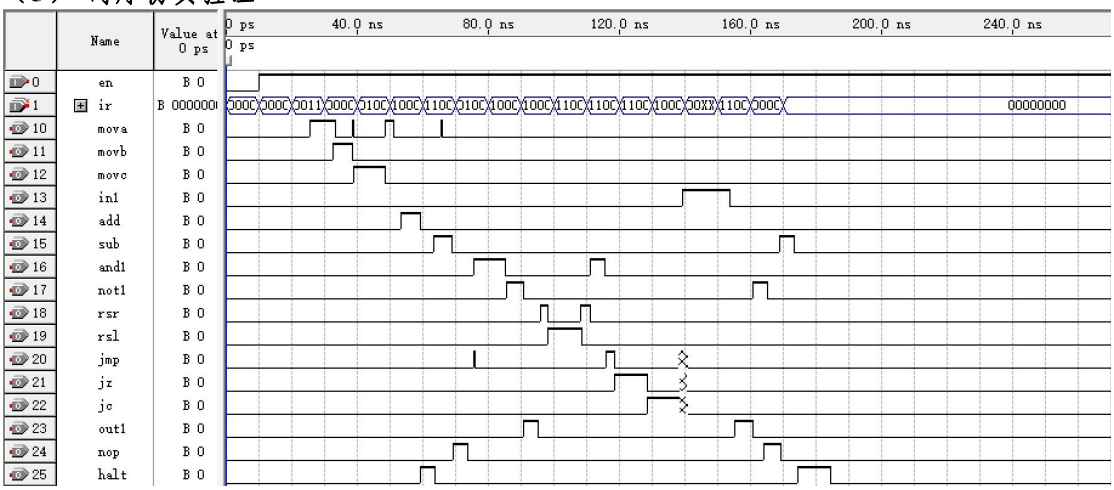
1、由视图可得，视图左边为输入，右边为输出。

2、其中连接有一系列的元器件。比如比较器：当输入相等时输出 1，不相等时输出 0；

3、还有许多 2-1 选择器，当控制信号为 0 时，输出第一位，控制信号为 1 时，输出第二位。图中输入信号为 ir 和 en，输出信号包 括 add 等 16 种情况。各个输出端口之间通过导线相连。

结论：一个功能的实现需要经过多重门的处理后才能实现，一个元件的内部原理结构图十分复杂。

（5） 时序仿真验证



结果分析及结论:

分析：

1、时序仿真是指在布线后进行，是最接近真实器件运行的仿真，它与特定的器件有关，又包含了器件和布线的延时信息。

2、由波形可得，当输入状态发生改变时，输出结果并未同时改变，而是有一定延迟，同时由于输入状态的改变，导致电路出现“冒险”，导致输出结果并未与预期结果相同。

结论：

1、时序仿真可以用来验证程序在目标器件中的时序关系。同时考虑了器件的延迟后，其输出结果跟接近实际情况，但是考虑的情况过多，不容易操作，容易产生错误。

2、时序仿真不仅反应出输出和输入的逻辑关系，同时还计算了时间的延时信息，是与实际系统更接近的一种仿真结果。不过，要注意的是，这个时间延时是仿真软件“估算”出来的。

（6） 时序分析

tpd

	Slack	Required P2P Time	Actual P2P Time	From	To
1	N/A	None	16.272 ns	ir[7]	movc
2	N/A	None	16.055 ns	ir[5]	movc
3	N/A	None	16.052 ns	ir[6]	movc
4	N/A	None	15.903 ns	ir[7]	rsr
5	N/A	None	15.880 ns	ir[7]	jmp
6	N/A	None	15.848 ns	ir[7]	rsl
7	N/A	None	15.830 ns	ir[5]	jmp
8	N/A	None	15.792 ns	ir[5]	rsr
9	N/A	None	15.785 ns	ir[7]	mova
10	N/A	None	15.777 ns	ir[7]	jc
11	N/A	None	15.737 ns	ir[5]	rsl
12	N/A	None	15.727 ns	ir[5]	jc
13	N/A	None	15.649 ns	ir[6]	jmp
14	N/A	None	15.614 ns	ir[7]	and1
15	N/A	None	15.592 ns	ir[6]	rsr
16	N/A	None	15.568 ns	ir[5]	mova
17	N/A	None	15.565 ns	ir[6]	mova
18	N/A	None	15.564 ns	ir[7]	jz
19	N/A	None	15.546 ns	ir[6]	jc
20	N/A	None	15.537 ns	ir[6]	rsl
21	N/A	None	15.524 ns	ir[5]	and1
22	N/A	None	15.514 ns	ir[5]	jz
23	N/A	None	15.511 ns	ir[7]	not1
24	N/A	None	15.355 ns	ir[7]	out1
25	N/A	None	15.344 ns	ir[6]	not1
26	N/A	None	15.341 ns	ir[7]	movb
27	N/A	None	15.340 ns	ir[5]	not1
28	N/A	None	15.339 ns	ir[6]	and1
29	N/A	None	15.333 ns	ir[6]	jz
30	N/A	None	15.178 ns	ir[6]	out1
31	N/A	None	15.176 ns	ir[5]	out1
32	N/A	None	15.124 ns	ir[5]	movb
33	N/A	None	15.121 ns	ir[6]	movb
34	N/A	None	14.598 ns	ir[7]	halt
35	N/A	None	14.341 ns	en	halt
36	N/A	None	14.234 ns	en	movc
37	N/A	None	14.072 ns	ir[7]	nop
38	N/A	None	14.008 ns	ir[5]	nop
39	N/A	None	13.887 ns	ir[6]	nop
40	N/A	None	13.886 ns	ir[7]	in1
41	N/A	None	13.854 ns	ir[7]	add
42	N/A	None	13.849 ns	ir[2]	movc
43	N/A	None	13.820 ns	ir[5]	in1
44	N/A	None	13.813 ns	ir[5]	halt
45	N/A	None	13.726 ns	ir[6]	halt
46	N/A	None	13.637 ns	ir[5]	add
47	N/A	None	13.635 ns	ir[2]	mova
48	N/A	None	13.617 ns	ir[6]	in1
49	N/A	None	13.556 ns	ir[6]	add
50	N/A	None	13.541 ns	en	jc
51	N/A	None	13.527 ns	ir[7]	sub
52	N/A	None	13.460 ns	ir[5]	sub
53	N/A	None	13.388 ns	en	and1
54	N/A	None	13.355 ns	ir[6]	sub
55	N/A	None	13.153 ns	en	not1
56	N/A	None	13.153 ns	en	movb
57	N/A	None	13.150 ns	en	mova
58	N/A	None	13.110 ns	en	nop
59	N/A	None	13.110 ns	en	out1
60	N/A	None	13.068 ns	en	in1
61	N/A	None	13.034 ns	en	add
62	N/A	None	12.817 ns	en	jmp
63	N/A	None	12.803 ns	ir[2]	movb
64	N/A	None	12.781 ns	en	rsr
65	N/A	None	12.752 ns	en	jz
66	N/A	None	12.716 ns	en	rsl
67	N/A	None	12.711 ns	en	sub
68	N/A	None	11.701 ns	ir[4]	movc
69	N/A	None	11.324 ns	ir[4]	jmp
70	N/A	None	11.316 ns	ir[4]	rsr
71	N/A	None	11.261 ns	ir[4]	rsl
72	N/A	None	11.221 ns	ir[4]	jc
73	N/A	None	11.214 ns	ir[4]	mova
74	N/A	None	11.054 ns	ir[4]	and1
75	N/A	None	11.008 ns	ir[4]	jz
76	N/A	None	10.948 ns	ir[4]	not1
77	N/A	None	10.783 ns	ir[4]	out1
78	N/A	None	10.770 ns	ir[4]	movb
79	N/A	None	9.488 ns	ir[4]	nop
80	N/A	None	9.459 ns	ir[4]	halt
81	N/A	None	9.302 ns	ir[4]	in1
82	N/A	None	9.283 ns	ir[4]	add
81	N/A	None	9.302 ns	ir[4]	in1
82	N/A	None	9.283 ns	ir[4]	add
83	N/A	None	9.133 ns	ir[3]	movc
84	N/A	None	9.094 ns	ir[0]	mova
85	N/A	None	9.011 ns	ir[0]	movc
86	N/A	None	8.949 ns	ir[4]	sub
87	N/A	None	8.825 ns	ir[1]	mova
88	N/A	None	8.787 ns	ir[1]	movc
89	N/A	None	8.694 ns	ir[1]	jc
90	N/A	None	8.613 ns	ir[3]	mova
91	N/A	None	8.524 ns	ir[0]	jz
92	N/A	None	8.524 ns	ir[3]	movb
93	N/A	None	8.494 ns	ir[0]	jmp
94	N/A	None	8.478 ns	ir[0]	rsl
95	N/A	None	8.451 ns	ir[0]	rsr
96	N/A	None	8.358 ns	ir[1]	jmp

结果分析及结论:

由图可得，Timing Analyzer Summmary 总结所有经典定时分析的结果，并报告每个 定时特性的最坏情况定时。比如从 ir[3]到 jmp 的最坏定时情况的 tpd 为 11.720ns。下面的 tpd 报告表则给出了源节点和目标节点之间的 tpd 延迟时间，比如第二行中 ir[5]到 add 的 tpd 为 11.699ns。实际连接图中个元器件连接之间是存在时间延迟的，而且不同的元器件之间的时间延迟也不相同。

3.2.2ALU

（1）接口设计

```
input [7:0] a;
input [7:0] b;
input m;
input [3:0] s;
output cf,zf;
output [7:0] t;
```

（2）功能实现

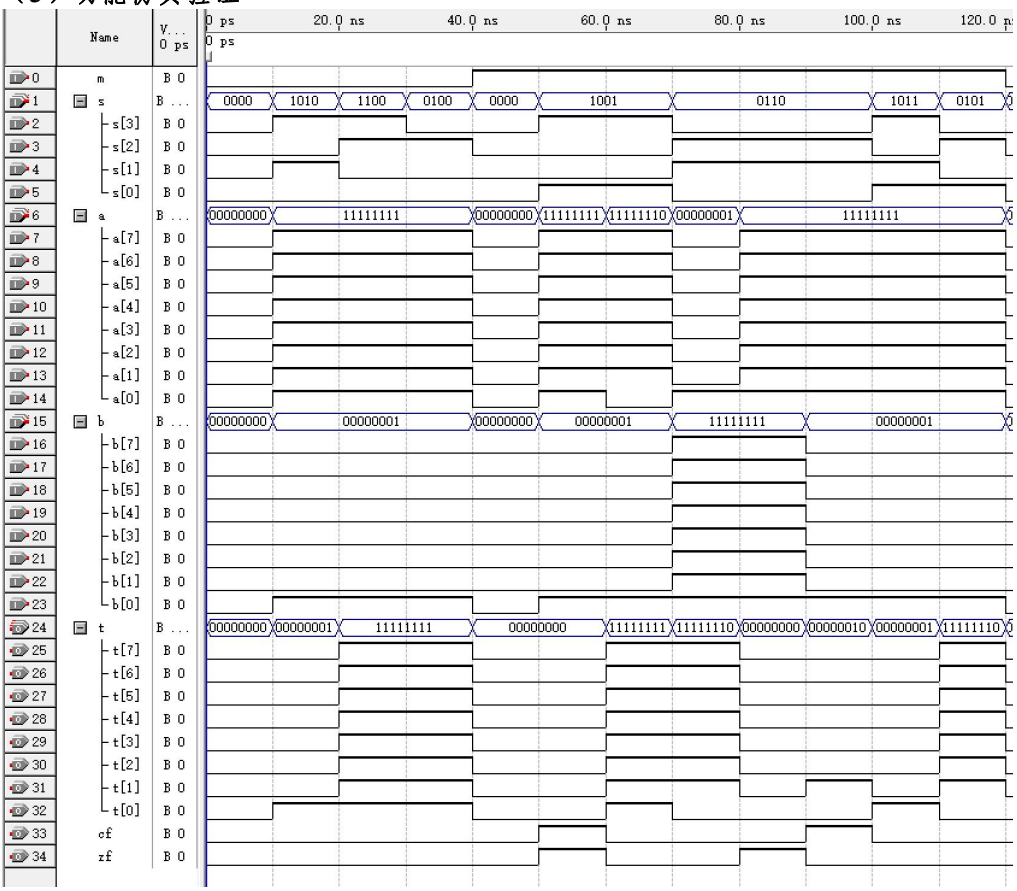
算术逻辑运算类指令：

ADD R1, R2　　SUB R1, R2　　AND R1, R2　　NOT R1

这类指令的执行过程为：

由 R2 的编码通过 RAA1、RAA0 从通用寄存器组 A 口读出 R2 的内容,由 R1 的编码通过 RWBA1、RWBA0 从通用寄存器组 B 口读出 R1 的内容，在 S3～S0 和 M 的控制下，实现运算，经移位逻辑送入总线 BUS；由/WE 控制和 R1 的编码选择 RWBA1、RWBA0，将 BUS 上的数据写入通用寄存器 R1。其中 ADD 和 SUB 指令影响状态位 Cf 和 Zf。

（3）功能仿真验证



结果分析及结论:

分析：功能仿真是指不考虑器件延时和布线延时的理想情况下对源代码进行逻辑功能的验证。由仿真波形可得，对于输入状态的变化，输出结果实时变化，没有延迟，其结果与电路设计的真值表的结果相对应。

当控制信号 m 为 1，s 为 1001 时，执行 t=a+b

当控制信号 m 为 1，s 为 0110 时，执行 t=b-a

当控制信号 m 为 1，s 为 1011 时，执行 t=a&b

当控制信号 m 为 1，s 为 0101 时，执行 t=~b

当控制信号 m 为 1，s 为 1010 时，执行 t=b

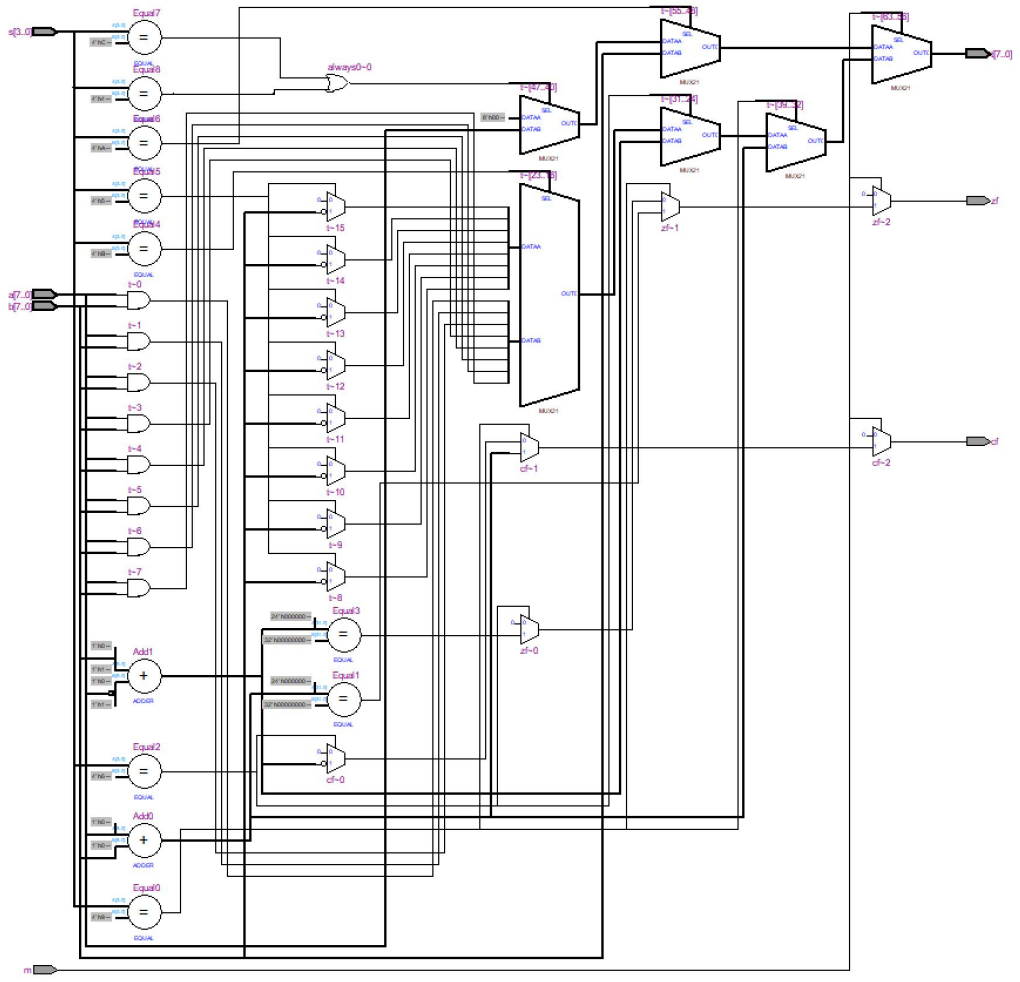
当控制信号 m 为 0，s 为 1100 时，执行 t=a

有进位和借位时 cf 为 1，否则为 0；

和为 0 或差为 0 时 zf 为 1，否则为 0；

结论：功能仿真操作简单，能体现和验证实验的功能，但忽略延迟的影响会使结果与实际结果有一定误差。

（4）RTL 视图



结果分析及结论：

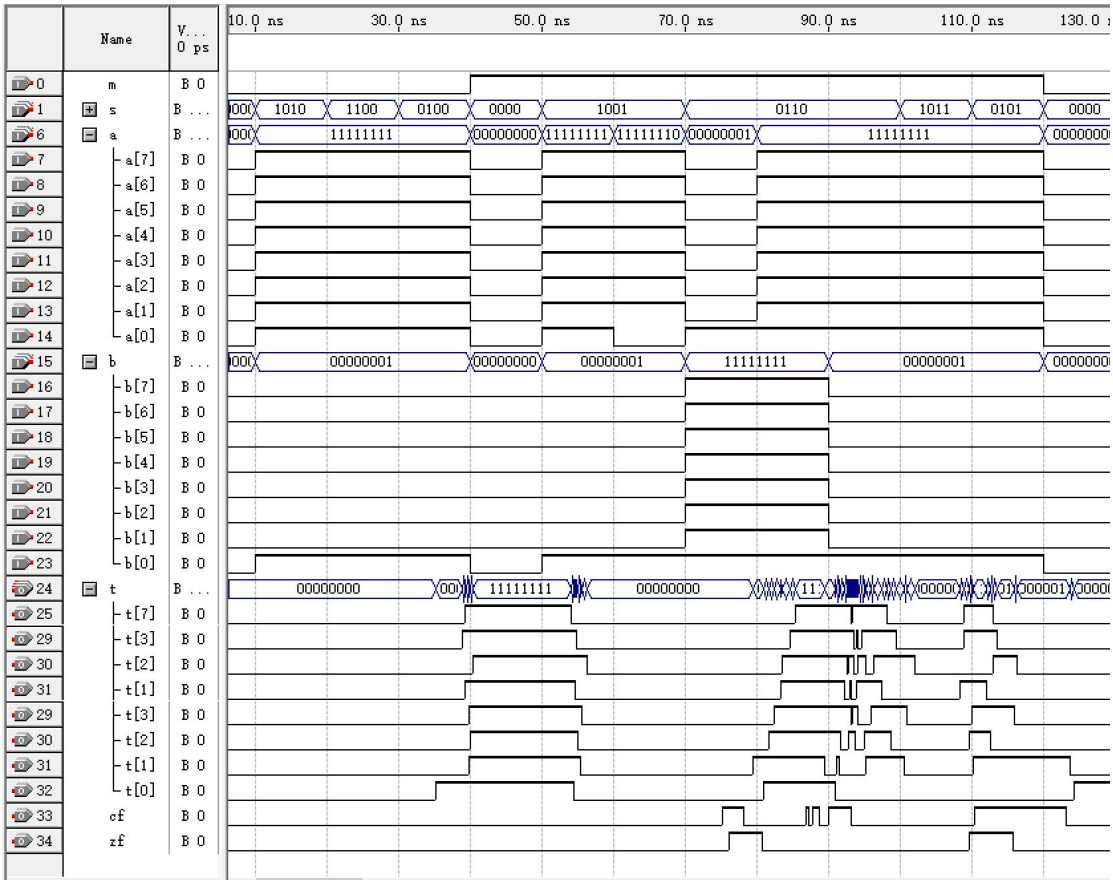
分析：

1、由视图可得，视图左边为输入，右边为输出。其中连接有一系列的元器件。比如比较器：当输入相等时输出 1，不相等时输出 0；

2、同时存在大量的 2-1 选择器，当控制信号为 0 时，输出第一位，控制信号为 1 时，输出第二位。图中输入信号为 m，s，a，b，输出信号 为 t，cf，zf。各个输出端口之间通过导线相连。

结论：一个功能的实现需要经过多重门的处理后才能实现，一个元件的内部原理结构图十分复杂。

（5）时序仿真验证



结果分析及结论:

分析：

1、时序仿真是指在布线后进行，是最接近真实器件运行的仿真，它与特定的器件有关，又包含了器件和布线的延时信息。

2、由波形可得，当输入状态发生改变时，输出结果并未同时改变，而是有一定延迟，同时由于输入状态的改变，导致电路出现“冒险”，导致输出结果并未与预期结果相同。

结论：

1、时序仿真可以用来验证程序在目标器件中的时序关系。同时考虑了器件的延迟后，其输出结果跟接近实际情况，但是考虑的情况过多，不容易操作，容易产生错误。

2、时序仿真不 仅反应出输出和输入的逻辑关系，同时还计算了时间的延时信息，是与实际系统更接近的一种仿真结果。不过，要注意的是，这个时间延时是仿真软件“估算”出来的。

（6）时序分析

Timing Analyzer Summary

	Type	Slack	Required Time	Actual Time	From	To	From Clock	To Clock	Failed Paths
1	Worst-case tpd	N/A	None	54.000 ns	b[0]	zf	--	--	0
2	Total number of failed paths								0

tpd

	Slack	Required P2P Time	Actual P2P Time	From	To
1	N/A	None	54.000 ns	b[0]	zf
2	N/A	None	53.500 ns	a[0]	zf
3	N/A	None	52.200 ns	a[1]	zf
4	N/A	None	51.800 ns	b[1]	zf
5	N/A	None	49.000 ns	b[2]	zf
6	N/A	None	48.000 ns	a[2]	zf
7	N/A	None	47.100 ns	b[0]	cf
8	N/A	None	46.600 ns	a[0]	cf
9	N/A	None	45.600 ns	b[0]	t[7]
10	N/A	None	45.300 ns	a[1]	cf
11	N/A	None	45.100 ns	a[0]	t[7]
12	N/A	None	45.000 ns	b[1]	cf
13	N/A	None	43.900 ns	a[3]	zf
14	N/A	None	43.800 ns	a[1]	t[7]
15	N/A	None	43.500 ns	b[1]	t[7]
16	N/A	None	43.100 ns	b[3]	zf
17	N/A	None	42.100 ns	b[2]	cf
18	N/A	None	41.600 ns	b[0]	t[6]
19	N/A	None	41.100 ns	a[2]	cf
20	N/A	None	41.100 ns	a[0]	t[6]
21	N/A	None	40.900 ns	b[0]	t[5]
22	N/A	None	40.600 ns	b[2]	t[7]
23	N/A	None	40.400 ns	a[0]	t[5]
24	N/A	None	39.800 ns	a[1]	t[6]
25	N/A	None	39.600 ns	a[2]	t[7]
26	N/A	None	39.500 ns	b[1]	t[6]
27	N/A	None	39.400 ns	b[4]	zf
28	N/A	None	39.100 ns	a[1]	t[6]
29	N/A	None	38.800 ns	b[1]	t[5]
30	N/A	None	38.400 ns	a[4]	zf
31	N/A	None	37.000 ns	a[3]	cf
32	N/A	None	36.600 ns	b[2]	t[6]
33	N/A	None	36.200 ns	b[3]	cf
34	N/A	None	36.000 ns	b[5]	zf
35	N/A	None	35.800 ns	b[2]	t[5]
36	N/A	None	35.600 ns	a[2]	t[6]
37	N/A	None	35.500 ns	a[3]	t[7]
38	N/A	None	35.100 ns	a[5]	zf
39	N/A	None	34.900 ns	a[2]	t[5]
40	N/A	None	34.700 ns	b[3]	t[7]
41	N/A	None	33.000 ns	b[0]	t[4]
42	N/A	None	32.500 ns	b[4]	cf
43	N/A	None	32.500 ns	a[0]	t[4]
44	N/A	None	31.800 ns	a[6]	zf
45	N/A	None	31.500 ns	a[4]	cf
46	N/A	None	31.500 ns	a[3]	t[6]
47	N/A	None	31.300 ns	b[6]	zf
48	N/A	None	31.200 ns	a[1]	t[4]
49	N/A	None	31.000 ns	b[4]	t[7]
50	N/A	None	31.000 ns	b[0]	t[3]
51	N/A	None	30.900 ns	b[1]	t[4]
52	N/A	None	30.800 ns	a[3]	t[5]
53	N/A	None	30.700 ns	b[3]	t[6]
54	N/A	None	30.600 ns	a[0]	t[4]
55	N/A	None	30.000 ns	a[4]	t[7]
56	N/A	None	30.000 ns	b[3]	t[5]
57	N/A	None	29.800 ns	s[3]	zf
58	N/A	None	29.200 ns	s[1]	zf
59	N/A	None	29.200 ns	a[1]	t[3]
60	N/A	None	29.100 ns	b[5]	cf
61	N/A	None	28.900 ns	b[1]	t[3]
62	N/A	None	28.500 ns	s[3]	t[5]
63	N/A	None	28.200 ns	a[5]	cf
64	N/A	None	28.000 ns	s[0]	zf
65	N/A	None	28.000 ns	b[2]	t[4]
66	N/A	None	27.800 ns	s[2]	zf
67	N/A	None	27.800 ns	s[3]	t[3]
68	N/A	None	27.600 ns	b[5]	t[7]
69	N/A	None	27.600 ns	s[3]	t[1]
70	N/A	None	27.500 ns	s[1]	t[5]
71	N/A	None	27.300 ns	b[7]	zf
72	N/A	None	27.300 ns	s[3]	t[0]
73	N/A	None	27.100 ns	s[3]	t[2]
74	N/A	None	27.000 ns	s[3]	t[6]
75	N/A	None	27.000 ns	b[4]	t[6]
76	N/A	None	27.000 ns	a[2]	t[4]
77	N/A	None	26.800 ns	s[1]	t[3]
78	N/A	None	26.700 ns	a[5]	t[7]
79	N/A	None	26.700 ns	s[0]	t[5]
80	N/A	None	26.700 ns	s[3]	t[4]
81	N/A	None	26.700 ns	s[1]	t[1]
82	N/A	None	26.600 ns	s[1]	t[2]
83	N/A	None	26.500 ns	a[7]	zf
84	N/A	None	26.500 ns	s[3]	t[7]
85	N/A	None	26.500 ns	s[2]	t[5]
86	N/A	None	26.300 ns	b[4]	t[5]
87	N/A	None	26.300 ns	s[1]	t[0]
88	N/A	None	26.000 ns	s[1]	t[6]
89	N/A	None	26.000 ns	a[4]	t[6]
90	N/A	None	26.000 ns	s[0]	t[3]
91	N/A	None	26.000 ns	b[2]	t[3]
92	N/A	None	25.900 ns	s[1]	t[7]
93	N/A	None	25.900 ns	s[1]	t[4]
94	N/A	None	25.800 ns	s[2]	t[3]
95	N/A	None	25.800 ns	s[0]	t[1]
96	N/A	None	25.600 ns	s[2]	t[1]
97	N/A	None	25.500 ns	s[0]	t[0]
98	N/A	None	25.300 ns	s[3]	cf
99	N/A	None	25.300 ns	a[4]	t[5]
100	N/A	None	25.300 ns	s[2]	t[0]
101	N/A	None	25.200 ns	s[0]	t[6]
102	N/A	None	24.900 ns	s[0]	t[4]
103	N/A	None	24.700 ns	s[1]	cf
104	N/A	None	24.700 ns	s[0]	t[7]
105	N/A	None	24.700 ns	s[2]	t[4]
106	N/A	None	24.700 ns	b[0]	t[2]
107	N/A	None	24.600 ns	s[2]	t[2]
108	N/A	None	24.500 ns	s[2]	t[7]
109	N/A	None	24.400 ns	b[0]	cf
110	N/A	None	24.200 ns	a[0]	t[0]
111	N/A	None	23.800 ns	s[0]	t[2]
112	N/A	None	23.600 ns	b[6]	t[6]
113	N/A	None	23.500 ns	m	zf
114	N/A	None	23.500 ns	s[0]	cf
115	N/A	None	23.500 ns	a[6]	t[7]
116	N/A	None	23.300 ns	s[2]	cf
117	N/A	None	23.100 ns	b[6]	t[7]
118	N/A	None	23.100 ns	m[0]	t[0]
119	N/A	None	23.100 ns	a[3]	t[4]
120	N/A	None	23.000 ns	b[2]	t[2]
121	N/A	None	22.900 ns	b[7]	t[7]
122	N/A	None	22.900 ns	b[5]	t[5]
123	N/A	None	22.900 ns	m[1]	t[2]
124	N/A	None	22.700 ns	a[5]	t[6]
125	N/A	None	22.700 ns	b[6]	t[6]
126	N/A	None	22.600 ns	b[1]	t[2]
127	N/A	None	22.500 ns	m	t[6]
128	N/A	None	22.500 ns	m[3]	t[3]
129	N/A	None	22.300 ns	b[4]	t[4]
130	N/A	None	22.300 ns	b[0]	t[1]
131	N/A	None	22.200 ns	a[7]	t[7]
132	N/A	None	22.100 ns	a[5]	t[5]
133	N/A	None	22.100 ns	b[3]	t[4]
134	N/A	None	22.000 ns	m[2]	t[0]
135	N/A	None	21.800 ns	m	t[3]
136	N/A	None	21.800 ns	a[0]	t[1]
137	N/A	None	21.600 ns	m	t[1]
138	N/A	None	21.500 ns	b[3]	t[3]
139	N/A	None	21.300 ns	m[4]	t[4]
140	N/A	None	21.300 ns	m	t[2]
141	N/A	None	21.300 ns	m	t[0]
142	N/A	None	21.000 ns	m	t[6]
143	N/A	None	20.700 ns	m	t[4]
144	N/A	None	20.600 ns	m	t[7]
145	N/A	None	20.600 ns	m[1]	t[1]
146	N/A	None	20.400 ns	b[7]	cf
147	N/A	None	20.200 ns	b[1]	t[1]
148	N/A	None	20.000 ns	b[0]	t[0]
149	N/A	None	20.000 ns	m[0]	t[0]
150	N/A	None	19.600 ns	m[7]	cf
151	N/A	None	19.200 ns	m	cf

结果分析及结论:

由图可得，Timing Analyzer Summmary 总结所有经典定时分析的结果，并报告每个 定时特性的最坏情况定时。比如从 a[0]到 zf 的最坏定时情况的 tpd 为 38.50ns。下面的 tpd 报告表则给出了源节点和目标节点之间的 tpd 延迟时间,比如第二行中 s[0]到 t[0]的 tpd 为 29.6ns。实际连接图中个元器件连接之间是存在时间延迟的，而且不同的元器件之间的时间延迟也不同。

3.2.3 8 重 3-1 多路复用器

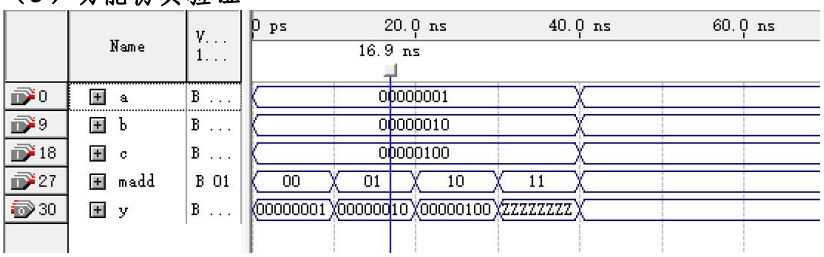
（1）接口设计

```
input [7:0] a;
input [7:0] b;
input [7:0] c;
input [1:0] madd;
output [7:0] y;
```

（2）功能实现

多路复用器是一个组合电路，它可以从多个输入中选择一个输入，并将信息直接传输到输出。选择哪一条输入线由一组输入变量控制，它们被称为选择输入。通常，2^n 条输入线要 n 个选择输入，选择输入的位组合决定选择哪个输入线。

（3）功能仿真验证



结果分析及结论：

分析：

功能仿真是指不考虑器件延时和布线延时的理想情况下对源代码进行逻辑功能的验证。由仿真波形可得，对于输入状态的变化，输出结果实时变化，没有延迟，其结果与电路设计的真值表的结果相对应。

madd=00 时，控制输出 y 等于 a，正确

madd=01 时，控制输出 y 等于 b，正确

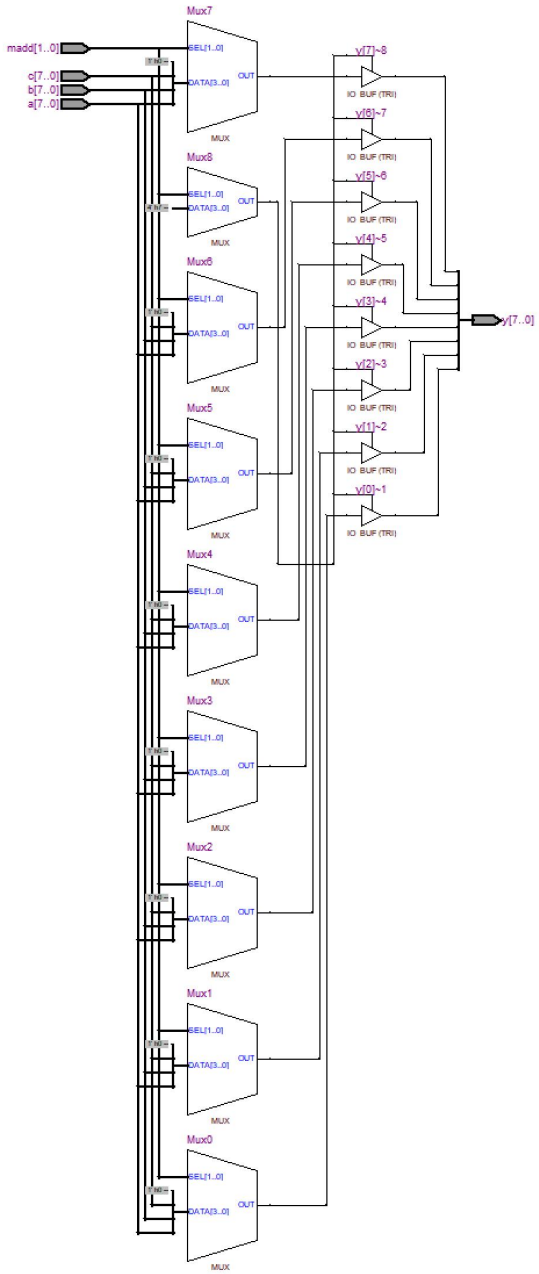
Madd=10 时，控制输出 y 等于 c，正确

Madd=11 时，控制输出高阻态，正确

结论：

功能仿真操作简单，能体现和验证实验的功能，但忽略延迟的影响会使结果与实际结果有一定误差。

（4）RTL 视图

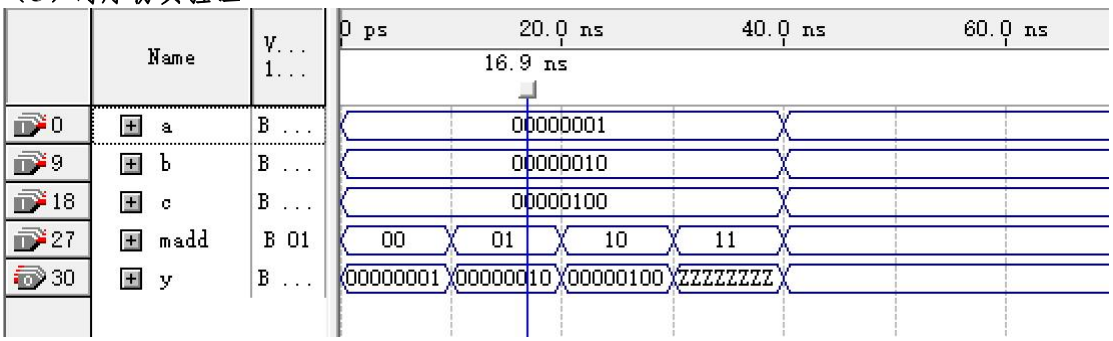


视图分析：

由视图可得，视图左边为输入，右边为输出。其中连接有一系列的元器件。比如比较器：当输入相等时输出 1，不相等时输出 0；还有大部分的 2-1 选择器构成，当控制信号为 0 时，输出第一位，控制信号为 1 时，输出第二位。图中输入信号为 madd 和 a，b，c，输出信号为 y。各个输出端口之间通过导线相连。

结论：一个功能的实现需要经过多重门的处理后才能实现，一个元件的内部原理结构图十分复杂。

（5）时序仿真验证



结果分析及结论：

分析：功能仿真是指不考虑器件延时和布线延时的理想情况下对源代码进行逻辑功能的验证。由仿真波形可得，对于输入状态的变化，输出结果实时变化，没有延迟，其结果与电路设计的真值表的结果相对应。

madd=00 时，控制输出 y 等于 a，正确

madd=01 时，控制输出 y 等于 b，正确

Madd=10 时，控制输出 y 等于 c，正确

Madd=11 时，控制输出高阻态，正确

结论：功能仿真操作简单，能体现和验证实验的功能，但忽略延迟的影响会使结果与实际结果有一定误差。

（6）时序分析

Timing Analyzer Summary

	Type	Slack	Required Time	Actual Time	From	To	From Clock	To Clock	Failed Paths
1	Worst-case tpd	N/A	None	15.082 ns	c[6]	y[6]	--	--	0
2	Total number of failed paths								0

tpd

	Slack	Required P2P Time	Actual P2P Time	From	To
1	N/A	None	15.082 ns	c[6]	y[6]
2	N/A	None	15.027 ns	a[6]	y[6]
3	N/A	None	13.820 ns	a[7]	y[7]
4	N/A	None	13.595 ns	madd[0]	y[3]
5	N/A	None	13.437 ns	madd[0]	y[7]
6	N/A	None	13.254 ns	c[7]	y[7]
7	N/A	None	13.191 ns	c[3]	y[3]
8	N/A	None	13.104 ns	a[5]	y[5]
9	N/A	None	13.101 ns	madd[0]	y[6]
10	N/A	None	13.046 ns	madd[0]	y[0]
11	N/A	None	13.030 ns	madd[0]	y[5]
12	N/A	None	12.962 ns	c[5]	y[5]
13	N/A	None	12.852 ns	b[7]	y[7]
14	N/A	None	12.840 ns	c[2]	y[2]
15	N/A	None	12.737 ns	madd[0]	y[1]
16	N/A	None	12.721 ns	madd[0]	y[4]
17	N/A	None	12.714 ns	a[1]	y[1]
18	N/A	None	12.695 ns	b[5]	y[5]
19	N/A	None	12.674 ns	a[2]	y[2]
20	N/A	None	12.403 ns	a[4]	y[4]
21	N/A	None	12.388 ns	a[3]	y[3]
22	N/A	None	12.330 ns	b[3]	y[3]
23	N/A	None	12.313 ns	madd[0]	y[2]
24	N/A	None	12.007 ns	b[4]	y[4]
25	N/A	None	11.875 ns	b[6]	y[6]
26	N/A	None	11.758 ns	c[4]	y[4]
27	N/A	None	11.689 ns	b[1]	y[1]
28	N/A	None	11.080 ns	c[1]	y[1]
29	N/A	None	10.581 ns	b[2]	y[2]
30	N/A	None	10.105 ns	madd[1]	y[6]
31	N/A	None	9.077 ns	madd[1]	y[3]
32	N/A	None	8.914 ns	madd[1]	y[7]
33	N/A	None	8.489 ns	a[0]	y[0]
34	N/A	None	8.474 ns	madd[1]	y[5]
35	N/A	None	8.420 ns	c[0]	y[0]
36	N/A	None	8.105 ns	madd[1]	y[0]
37	N/A	None	7.852 ns	madd[1]	y[2]
38	N/A	None	7.836 ns	b[0]	y[0]
39	N/A	None	7.772 ns	madd[1]	y[4]
40	N/A	None	7.765 ns	madd[1]	y[1]

结果分析及结论：

分析：

由图可得，Timing Analyzer Summmary 总结所有经典定时分析的结果，并报告每个定时特性的最坏情况定时。比如从 b[7]到 y[7]的最坏定时情况的 tpd 为 12.852ns。下面的 tpd 报告表则给出了源节点和目标节点之间的 tpd 延迟间，比如第二行中 a[7]到 y[7] 的 tpd 为 13.820ns。

结论：

实际连接图中个元器件连接之间是存在时间延迟的，而且不同的元器件之间的时间延迟也不相同。

3.2.4 移位逻辑

（1）接口设计

```
input fbus,flbus,frbus;
input [7:0] a;
output [7:0] w;
output cf;
```

（2）功能实现

移位指令：

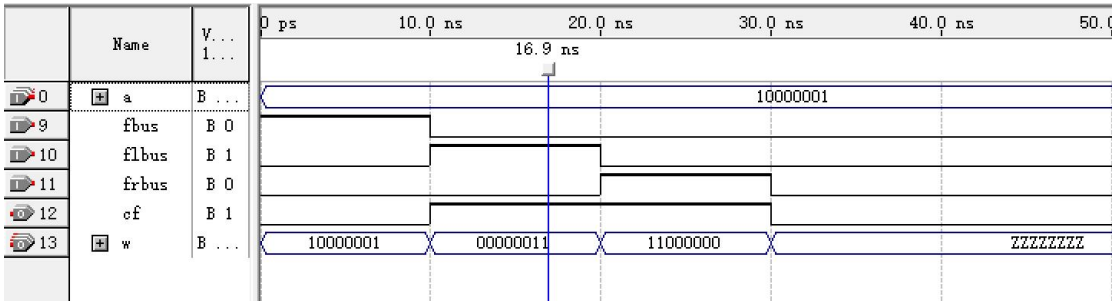
RSR R1

RSL R1

这类指令的执行过程为：

由 R1 的编码通过 RWBA1、RWBA0 从通用寄存器组 B 口读出 R1 的内容，在 S3~S0 和 M 的控制下通过 ALU,经移位逻辑循环右移或循环左移后送入总线 BUS；再由/WE 控制和 R1 的编码选择 RWBA1、RWBA0，将 BUS 上的数据写入通用寄存器 R1。

（3）功能仿真验证



结果分析及结论：

分析：

功能仿真是指不考虑器件延时和布线延时的理想情况下对源代码进行逻辑功能的验证。由仿真波形可得，对于输入状态的变化，输出结果实时变化，没有延迟，其结果与电路设计的真值表的结果相对应。

当 fbus=1，frbus=0，flbus=0，不执行移位操作，输出等于输入，cf 不改变

当 fbus=0，frbus=1，flbus=0，执行右移，输出等于输入右移移位，有进位的话 cf 为 1

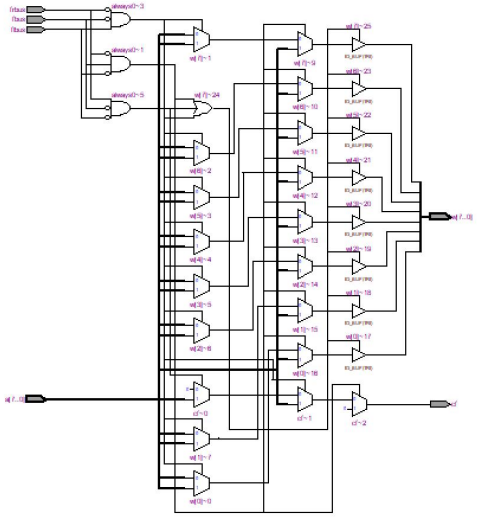
当 fbus=0，frbus=0，flbus=1，执行左移，输出等于输入左移一位，cf 不改变

当控制信号全为 0 时，输出为高阻态，正确

结论：

功能仿真操作简单，能体现和验证实验的功能，但忽略延迟的影响会使结果与实际结果有一定误差。

（4）RTL 视图

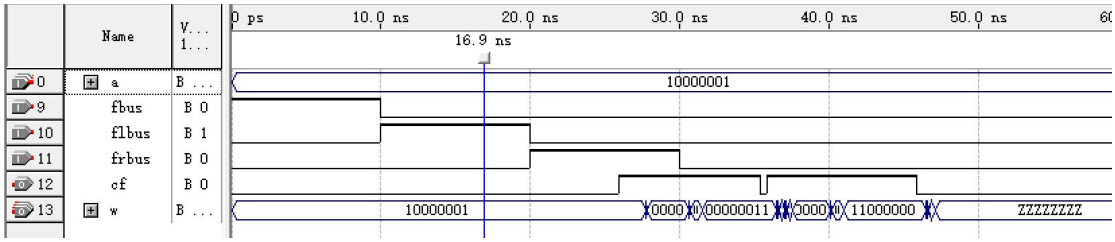


结果分析：

分析：由视图可得，视图左边为输入，右边为输出。其中连接有一系列的元器件。比如比较器：当输入相等时输出 1，不相等时输出 0；还有大部分的 2-1 选择器构成，当控制信号为 0 时，输出第一位，控制信号为 1 时，输出第二位。图中输入信号为 fbus，frbus，flbus 和 a，输出信号为 w。各个输出端口之间通过导线相连。

结论:一个功能的实现需要经过多重门的处理后才能实现,一个元件的内部原理结构图十分复杂。

（5）时序仿真验证



结果分析及结论：

分析：时序仿真是指在布线后进行，是最接近真实器件运行的仿真，它与特定的器件有关， 又包含了器件和布线的延时信息。由波形可得，当输入状态发生改变时，输出结果并未同时 改变，而是有一定延迟，同时由于输入状态的改变，导致电路出现“冒险”，导致输出结果并未与预期结果相同。

结论：时序仿真可以用来验证程序在目标器件中的时序关系。同时考虑了器件的延迟后，其输出结果跟接近实际情况，但是考虑的情况过多，不容易操作，容易产生错误。时序仿真不仅反应出输出和输入的逻辑关系，同时还计算了时间的延时信息，是与实际系统更接近的一种仿真结果。不过，要注意的是，这个时间延时是仿真软件“估算”出来的。

（6）时序分析

Timing Analyzer Summary

	Type	Slack	Required Time	Actual Time	From	To	From Clock	To Clock	Failed Paths
1	Worst-case tpd	N/A	None	21.700 ns	fbus	w[5]	--	--	0
2	Total number of failed paths								0

tpd

	Slack	Required P2P Time	Actual P2P Time	From	To
1	N/A	None	21.700 ns	flbus	w[5]
2	N/A	None	21.700 ns	fbus	w[5]
3	N/A	None	21.500 ns	flbus	w[1]
4	N/A	None	21.500 ns	fbus	w[1]
5	N/A	None	21.200 ns	frbus	w[5]
6	N/A	None	21.000 ns	frbus	w[1]
7	N/A	None	20.900 ns	flbus	w[3]
8	N/A	None	20.900 ns	fbus	w[3]
9	N/A	None	20.700 ns	flbus	w[7]
10	N/A	None	20.700 ns	fbus	w[7]
11	N/A	None	20.700 ns	flbus	w[2]
12	N/A	None	20.700 ns	fbus	w[2]
13	N/A	None	20.700 ns	flbus	w[0]
14	N/A	None	20.700 ns	fbus	w[0]
15	N/A	None	20.600 ns	flbus	w[6]
16	N/A	None	20.600 ns	fbus	w[6]
17	N/A	None	20.600 ns	flbus	w[4]
18	N/A	None	20.600 ns	fbus	w[4]
19	N/A	None	20.400 ns	frbus	w[3]
20	N/A	None	20.200 ns	frbus	w[7]
21	N/A	None	20.200 ns	frbus	w[2]
22	N/A	None	20.200 ns	frbus	w[0]
23	N/A	None	20.100 ns	frbus	w[6]
24	N/A	None	20.100 ns	frbus	w[4]
25	N/A	None	20.000 ns	a[4]	w[5]
26	N/A	None	19.400 ns	a[1]	w[2]
27	N/A	None	19.400 ns	a[1]	w[0]
28	N/A	None	19.200 ns	a[4]	w[3]
29	N/A	None	19.100 ns	a[5]	w[6]
30	N/A	None	19.100 ns	a[5]	w[4]
31	N/A	None	19.000 ns	a[6]	w[5]
32	N/A	None	18.500 ns	a[6]	w[7]
33	N/A	None	18.500 ns	a[3]	w[2]
34	N/A	None	17.900 ns	a[3]	w[4]
35	N/A	None	17.300 ns	a[5]	w[5]
36	N/A	None	17.300 ns	a[1]	w[1]
37	N/A	None	17.200 ns	a[0]	w[1]
35	N/A	None	17.100 ns	a[2]	w[1]
39	N/A	None	16.400 ns	a[0]	w[7]
40	N/A	None	16.400 ns	a[7]	w[0]
41	N/A	None	16.300 ns	a[2]	w[3]
42	N/A	None	16.100 ns	a[0]	cf
43	N/A	None	16.000 ns	a[7]	w[6]
44	N/A	None	16.000 ns	a[4]	w[4]
45	N/A	None	15.900 ns	frbus	cf
46	N/A	None	15.900 ns	flbus	cf
47	N/A	None	15.900 ns	fbus	cf
48	N/A	None	15.300 ns	a[3]	w[3]
49	N/A	None	15.000 ns	a[6]	w[6]
50	N/A	None	13.500 ns	a[7]	w[7]
51	N/A	None	13.500 ns	a[0]	w[0]
52	N/A	None	13.400 ns	a[2]	w[2]
53	N/A	None	12.700 ns	a[7]	cf

结果分析及结论：

分析：由图可得，Timing Analyzer Summmary 总结所有经典定时分析的结果，并报告每个定时特性的最坏情况定时。比如从 fbus 到 w[0] 的最坏定时情况的 tpd 为 20.700ns。下面的 tpd 报告表则给出了源节点和目标节点之间的 tpd 延迟时间，比如第二行中 fbus 到 w[7] 的 tpd 为 20.700ns。

结论：实际连接图中个元器件连接之间是存在时间延迟的，而且不同的元器件之间的时间延迟也不相同。

3.2.5 控制信号产生逻辑

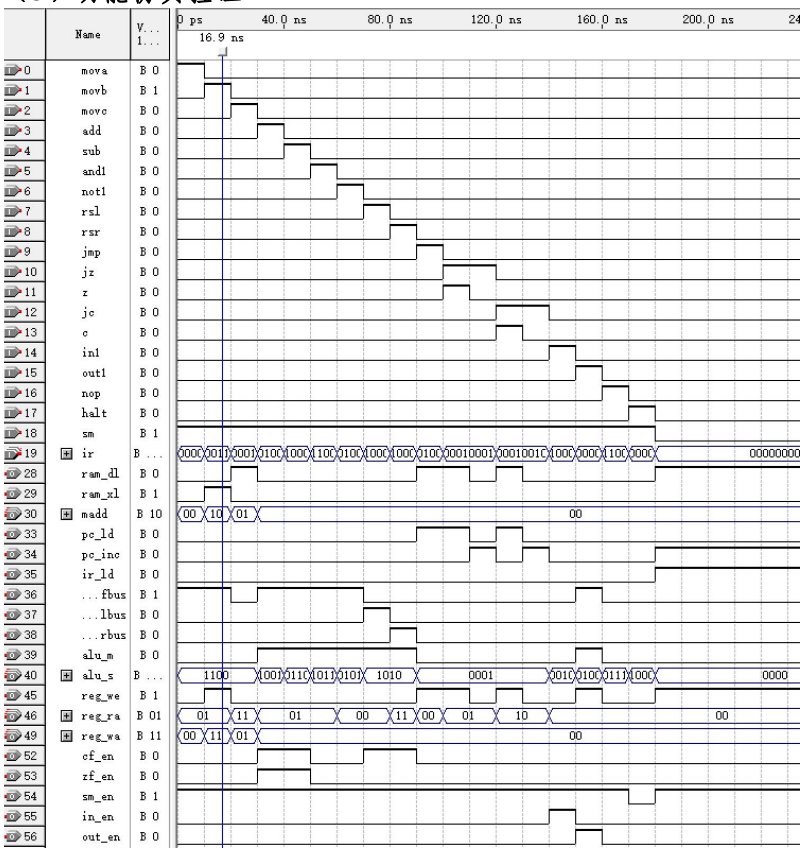
（1）接口设计

```
    input [7:0]ir;
    input
mova,movb,movc,add,sub,and1,not1,rsr,rsl,jmp,jz,z,jc,c,in1,out1,nop,halt,sm;
    output reg [1:0] reg_ra;
    output reg [1:0]reg_wa;
    output reg [1:0]madd;
    output reg [3:0]alu_s;
    output reg
pc_ld,pc_inc,reg_we,ram_xl,ram_dl,alu_m,shi_fbus,shi_frbus,shi_flbus,ir_ld,cf_en,zf
_en,sm_en,in_en,out_en;
```

（2）功能实现

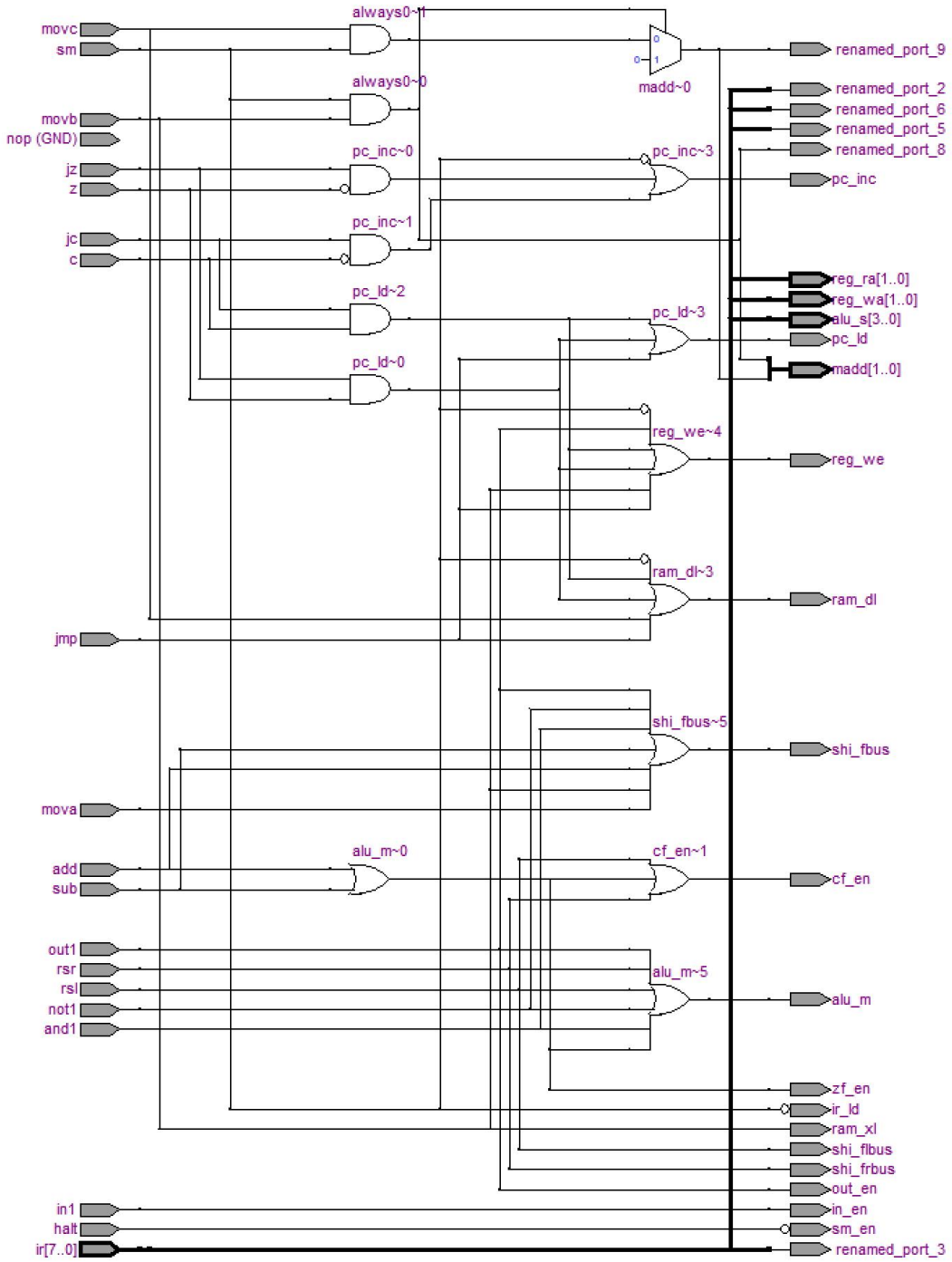
控制信号产生逻辑接收指令译码器的输出，在 SM、IR[7..0]以及状态位 Cf 和 Zf 的配合下产生每个模块所需要的控制信号。

（3）功能仿真验证



结果分析及结论：
分析：功能仿真是指不考虑器件延时和布线延时的理想情况下对源代码进行逻辑功能的验证。由仿真波形可得，对于输入状态的变化，输出结果实时变化，没有延迟，其结果与电路设计的真值表的结果相对应。
（1）当 mova 指令执行时，shi_fbus 和 sm_en 输出 1，其他输出为 0，madd 输出 00，alu_s 输出为 1100，reg_ra 输出 01，reg_wa 输出 00，正确
（2）当 movb 指令执行时，ram_xl 和 shi_fbus 和 reg_we 和 sm_en 输出为 1，其他输出为 0，madd 输出为 10，alu_s 输出为 1100，reg_ra 输出 01，reg_wa 输出 11，正确
（3）当 movc 指令执行时，ram_dl 和 sm_en 输出为 1，其他输出为 0，madd 输出 01，alu_s 输出 1100，reg_ra 输出 11，reg_wa 输出 01，正确
（4）当 add 指令执行时，shi_fbus，alu_en，cf_en，zf_en，sm_en 输出为 1，其他输出为 0，alu_s 为 1001，reg_ra 输出 01，reg_wa 输出 00，正确
（5）当 sub 指令执行时，shi_fbus 和 alu_m，cf_en，zf_en 和 sm_en 输出为 1，其他输出为 0，alu_s 输出 0110，reg_ra 输出 01，reg_wa 输出 00，正确
（6）当 and1 指令执行时，shi_fbus 和 alu_m 和 sm_en 输出 1，其他输出 0，alu_s 输出 1011，reg_ra 输出 01，reg_wa 输出 00，正确
（7）not1 指令执行时，shi_fbus 和 alu_m 和 sm_en 输出 1，其他输出 0，alu_s 输出 0101，reg_ra 输出 00，reg_wa 输出 00，正确
（8）rsl 指令执行时，shi_flbus 和 alu_m 和 cf_en 和 sm_en 输出 0，其他输出 0，alu_s 输出 1010，reg_ra 和 reg_wa 输出 00，正确
（9）rsr 指令执行时，shi_frbus 和 alu_m 和 cf_en 和 sm_en 输出 1，其他输出 0，alu_s 输出 10110，reg_ra 输出 11，reg_wa 输出 00，正确
（10）jmp 指令执行时，ram_dl，pc_ld，reg_we 和 sm_en 输出 1，其他输出 0，alu_s 输出 0001，reg_ra 和 reg_wa 输出 00，正确
（11）jz 指令为 1 和 jc 指令为 1 时，若 z 和 c 为 1 时，ram_dl 和 pc_ld 和 reg_we 和 sm_en 输出为 1，其他输出为 0，正确
若 z 和 c 为 0 时，pc_inc 和 reg_we 和 sm_en 输出 1，其他输出 0，正确
（12）in1 指令执行时，sm_en 和 in_en 输出 1，其他输出 0，正确
（13）out1 指令执行时，sm_en 和 out_en 输出 1，其他输出 0，正确
（14）nop 指令执行时，sm_en 输出 1，其他输出 0，正确
（15）halt 指令执行时，输出全为 0，正确
结论：功能仿真操作简单，能体现和验证实验的功能，但忽略延迟的影响会使结果与实际结果有一定误差。0，reg_ra 输出 11，reg_wa 输出 00，正确

（4）RTL 视图



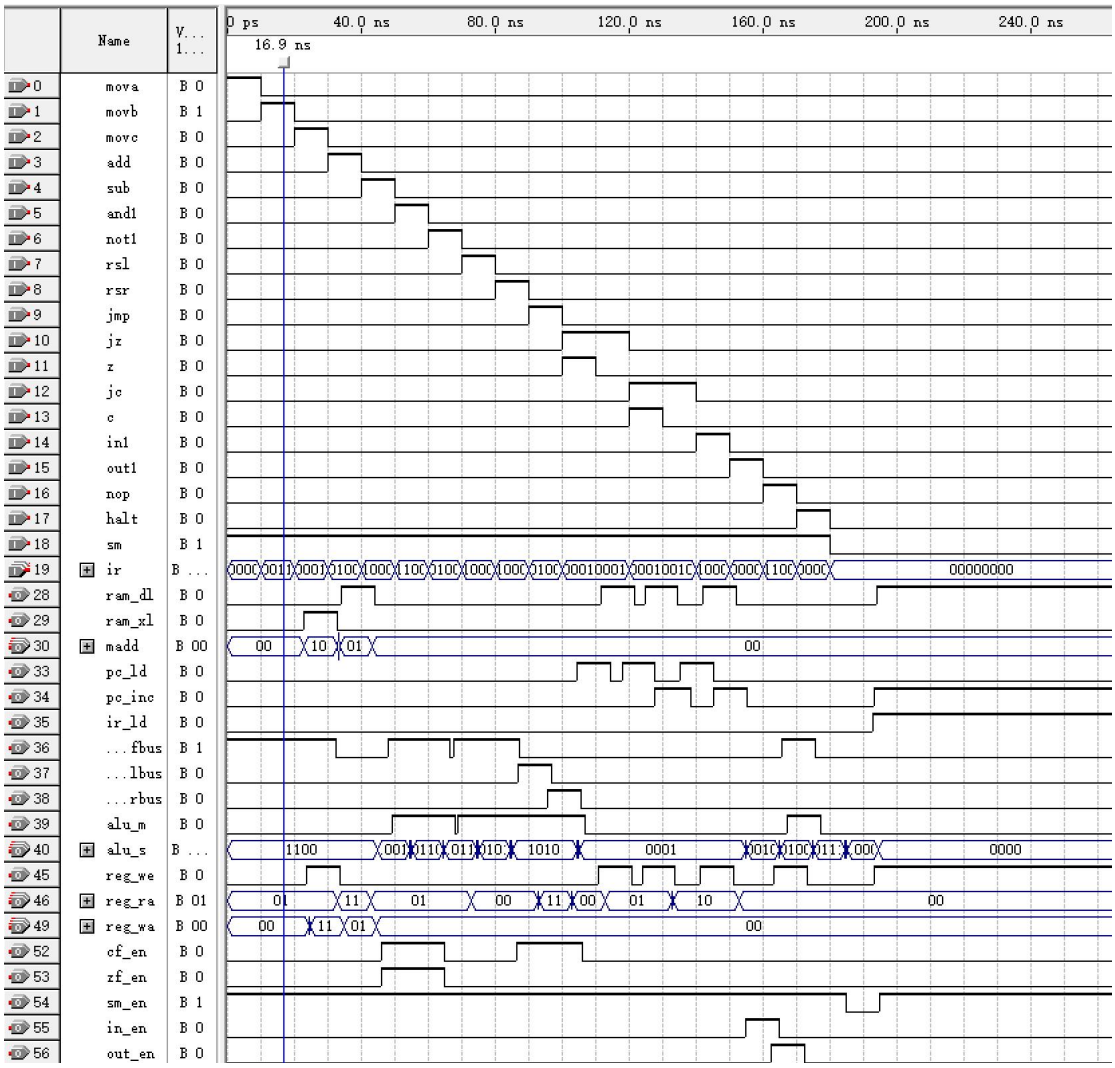
结果分析：

由视图可得，视图左边为输入，右边为输出。其中连接有一系列的元器件。比如比较器：当输入相等时输出 1，不相等时输出 0；还有大部分的与或门。图中输入信号为 sm 等 20 个，输出信号包括 reg_ra 等 19 种情况。各个输出端口之间通过导线相连。

结论：

一个功能的实现需要经过多重门的处理后才能实现，一个元件的内部原理结构图十分复杂。

（5）时序仿真验证



分析：

时序仿真是指在布线后进行，是最接近真实器件运行的仿真，它与特定的器件有关，又包含了器件和布线的延时信息。由波形可得，当输入状态发生改变时，输出结果并未同时改变，而是有一定延迟，同时由于输入状态的改变，导致电路出现“冒险”，导致输出结果并未与预期结果相同。

结论：

时序仿真可以用来验证程序在目标器件中的时序关系。同时考虑了器件的延迟后，其输出结果跟接近实际情况，但是考虑的情况过多，不容易操作，容易产生错误。时序仿真不仅反应出输出和输入的逻辑关系，同时还计算了时间的延时信息，是与实际系统更接近的一种仿真结果。不过，要注意的是，这个时间延时是仿真软件“估算”出来的。

（6）时序分析

Timing Analyzer Summary

	Type	Slack	Required Time	Actual Time	From	To	From Clock	To Clock	Failed Paths
1	Worst-case tpd	N/A	None	24.900 ns	jz	ram_dl	--	--	0
2	Total number of failed paths								0

tpd

	Slack	Required P2P Time	Actual P2P Time	From	To
1	N/A	None	24.900 ns	jz	ram_dl
2	N/A	None	24.400 ns	z	ram_dl
3	N/A	None	24.100 ns	jz	reg_we
4	N/A	None	23.600 ns	z	reg_we
5	N/A	None	22.000 ns	jc	ram_dl
6	N/A	None	22.000 ns	c	ram_dl
7	N/A	None	21.500 ns	jmp	ram_dl
8	N/A	None	21.200 ns	jc	reg_we
9	N/A	None	21.200 ns	c	reg_we
10	N/A	None	20.700 ns	jmp	reg_we
11	N/A	None	19.400 ns	add	alu_m
12	N/A	None	19.100 ns	and1	alu_m
13	N/A	None	18.600 ns	not1	alu_m
14	N/A	None	18.300 ns	jz	pc_inc
15	N/A	None	18.100 ns	sub	alu_m
16	N/A	None	18.000 ns	add	shi_fbus
17	N/A	None	18.000 ns	jz	pc_ld
18	N/A	None	17.800 ns	z	pc_inc
19	N/A	None	17.700 ns	and1	shi_fbus
20	N/A	None	17.500 ns	z	pc_ld
21	N/A	None	17.300 ns	rs1	alu_m
22	N/A	None	17.200 ns	not1	shi_fbus
23	N/A	None	17.100 ns	rs1	shi_flbus
24	N/A	None	17.100 ns	out1	alu_m
25	N/A	None	16.900 ns	rsr	alu_m
26	N/A	None	16.700 ns	sub	shi_fbus
27	N/A	None	16.500 ns	rs1	cf_en
28	N/A	None	16.200 ns	add	cf_en
29	N/A	None	16.100 ns	add	zf_en
30	N/A	None	16.100 ns	rsr	cf_en
31	N/A	None	16.100 ns	mova	shi_fbus
32	N/A	None	15.700 ns	out1	shi_fbus
33	N/A	None	15.600 ns	rsr	shi_frbus
34	N/A	None	15.500 ns	ir[2]	renamed_port_6
35	N/A	None	15.400 ns	jc	pc_inc
36	N/A	None	15.400 ns	c	pc_inc
37	N/A	None	15.100 ns	jc	pc_ld
38	N/A	None	15.100 ns	c	pc_ld
39	N/A	None	14.900 ns	ir[6]	alu_s[2]
40	N/A	None	14.900 ns	ir[4]	alu_s[0]
41	N/A	None	14.900 ns	ir[3]	reg_wa[1]
42	N/A	None	14.900 ns	in1	in_en
43	N/A	None	14.900 ns	halt	sm_en
44	N/A	None	14.900 ns	sub	cf_en
45	N/A	None	14.900 ns	ir[3]	renamed_port_5
46	N/A	None	14.800 ns	ir[5]	alu_s[1]
47	N/A	None	14.800 ns	sub	zf_en
48	N/A	None	14.600 ns	ir[7]	alu_s[3]
49	N/A	None	14.600 ns	ir[2]	reg_wa[0]
50	N/A	None	14.600 ns	jmp	pc_ld
51	N/A	None	14.400 ns	sm	renamed_port_8
52	N/A	None	14.400 ns	movb	renamed_port_9
53	N/A	None	14.200 ns	movc	ram_dl
54	N/A	None	14.200 ns	movc	renamed_port_9
55	N/A	None	13.900 ns	sm	ram_dl
56	N/A	None	13.900 ns	movb	renamed_port_8
57	N/A	None	13.900 ns	sm	renamed_port_9
58	N/A	None	13.600 ns	movb	madd[0]
59	N/A	None	13.600 ns	movb	reg_we
60	N/A	None	13.400 ns	sm	madd[1]
61	N/A	None	13.400 ns	movc	madd[0]
62	N/A	None	13.400 ns	out1	reg_we
63	N/A	None	13.100 ns	movb	madd[1]
64	N/A	None	13.100 ns	sm	madd[0]
65	N/A	None	13.100 ns	movb	ram_xl
66	N/A	None	13.100 ns	sm	reg_we
67	N/A	None	13.100 ns	sm	pc_inc
68	N/A	None	13.000 ns	ir[1]	reg_ra[1]
69	N/A	None	13.000 ns	ir[1]	renamed_port_2
70	N/A	None	12.900 ns	ir[0]	reg_ra[0]
71	N/A	None	12.900 ns	ir[0]	renamed_port_3
72	N/A	None	12.800 ns	sm	ir_ld
73	N/A	None	12.700 ns	movb	shi_fbus
74	N/A	None	12.400 ns	out1	out_en

结果分析及结论：

分析：由图可得，Timing Analyzer Summmary 总结所有经典定时分析的结果，并报告每个定时特性的最坏情况定时。比如从 sub 到 reg_we 的最坏定时情况的 tpd 为 26.600ns。下面 的 tpd 报告表则给出了源节点和目标节点之间的 tpd 延迟时间，比如第二行中 and1 到 reg_we 的 tpd 为 26.300ns。

结论：实际连接图中个元器件连接之间是存在时间延迟的，而且不同的元器件之间的时间延迟也不相同。

3.2.6 SM

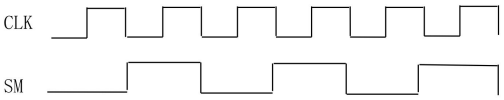
（1）接口设计

input clk,sm_en;

output reg sm;

（2）功能实现

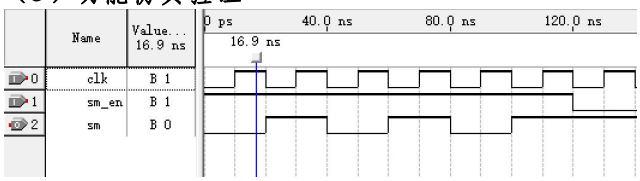
模型机中所有指令都要求两个周期完成，其中一个周期取指令，一个周期执行指令。如何区分当前周期是取指令还是执行指令呢？这就需要 SM 配合。SM 为 0 是取指令周期；SM 为 1 是执行指令周期。SM 的功能及封装如下：



SM的功能表：

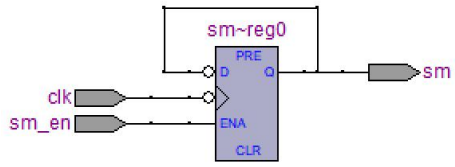
CLK	SM_EN	功能
⌐	1	SM⇦SM取反

（3）功能仿真验证

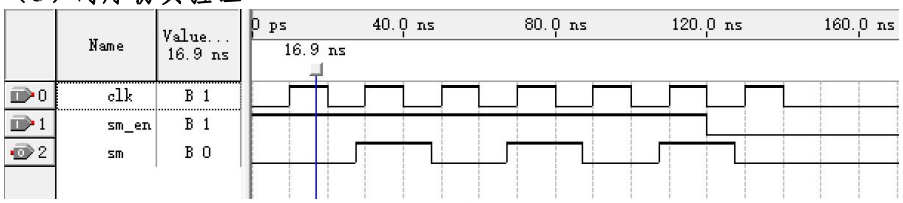


分析：对于功能仿真，可以看到当 sm_en 信号为 1 有效时,输出 sm 在时钟的下降沿发生翻转, 符号功能设计。当 sm_en 信号为 0 时，输出信号 sm 保持不变，正确。

（4）RTL 视图



（5）时序仿真验证



对于时序仿真，其输出结果和功能仿真类似，但存在 7ns 左右的延迟

结论：元件设计符合设计要求，元件内部存在 7ns 左右的延迟

3.2.7 指令寄存器 IR

（1）接口设计

input clk,ir_ld;

input [7:0] d;

output reg [7:0] ir;

（2）功能实现

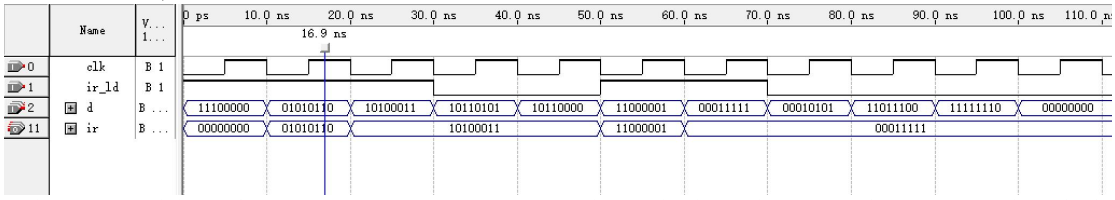
指令寄存器（IR，Instruction Register）用于暂存当前正在执行的指令。

指令寄存器将总线送来的指令存入 8 位寄存器中，但并不是每次总线上的数据都需要寄存，因为数据总线上有时传输指令，有时传输数据。当控制信号 IR_LD 为 1 时，指令寄存器在时钟信号 CLK 的下降沿将总线传输的指令写入寄存器。

指令寄存器 IR 是一个 8 位寄存器。其功能及封装如下：

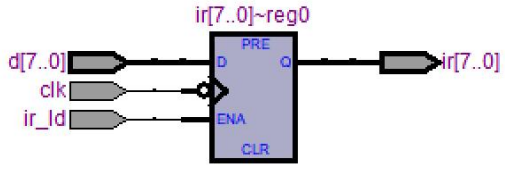
CLK	ir_ld	功能
⌐	1	d写入ir

（3）功能仿真验证

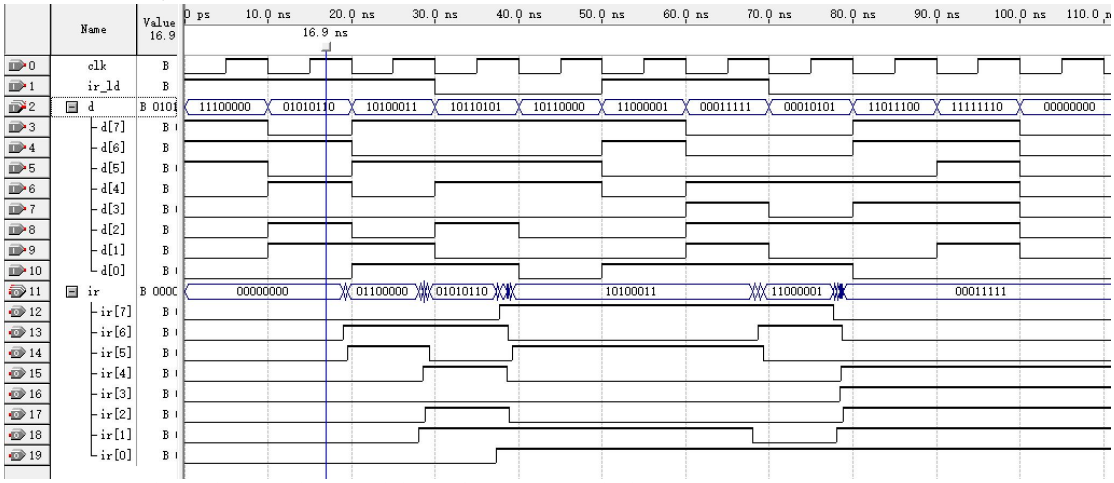


分析：对于功能仿真，可以看到当 sm_en 信号为 1 有效时,输出 sm 在时钟的下降沿发生翻转, 符号功能设计。当 sm_en 信号为 0 时，输出信号 sm 保持不变，正确。

（4）RTL 视图



（5）时序仿真验证



对于时序仿真，其输出结果和功能仿真类似，但存在 7ns 左右的延迟

结论：元件设计符合设计要求，元件内部存在 7ns 左右的延迟

3.2.8 状态寄存器 PSW

（1）接口设计

input clk,cf_en,zf_en,cf,zf;

output reg c,z;

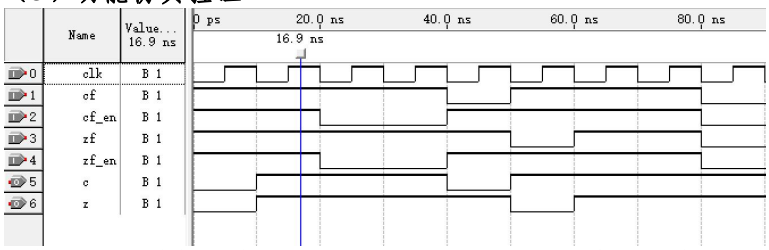
（2）功能实现

状态寄存器 PSW 是计算机系统的核心部件——运算器的扩展部分。本模型机

PSW 用来存放 ADD、SUB、RSR、RSL 指令执行结果的状态标志，如有无借位进位（C）、结果是否为零（Z）。有些机器也将 PSW 称为标志寄存器 FR（Flag Register）。 本模型机 PSW 是一个 2 位寄存器。其功能及封装如下：

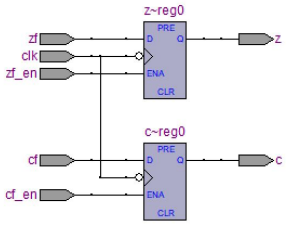
CLK	控制信号	功能
⌐	cf_en=1	cf写入c
⌐	zf_en=1	zf写入z

（3）功能仿真验证

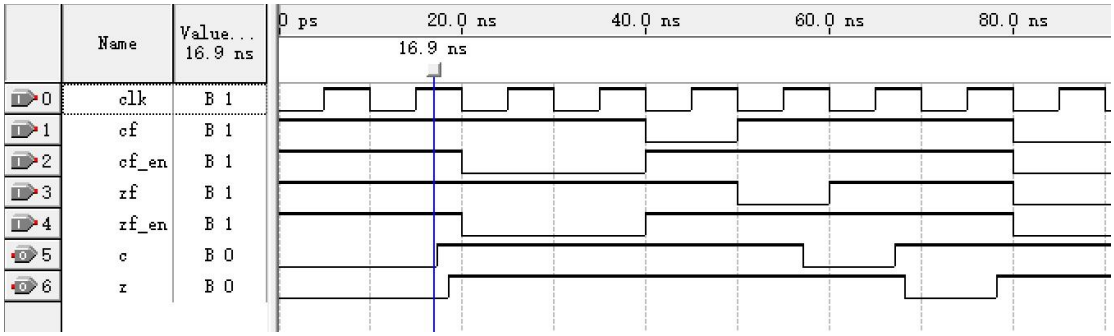


分析：对于功能仿真，0-20ns，cf_en 和 zf_en 为 1，在时钟下降沿，将 cf 和 zf 的值写入输出 c，z 中，20-40ns，cf_en 和 zf_en 为 0，输出 c 和 z 保持不变，正确

（4）RTL 视图



（5）时序仿真验证



对于时序仿真，其中输出 c 有 7ns 左右延迟 z 有 8ns 左右延迟，其输出变化与功能仿真大致相同。结论：元件设计符合要求，输出 c 有 7ns 延迟，输出 z 有 8ns 延迟。

3.2.9 指令计数器 PC

（1）接口设计

input clk,pc_ld,pc_inc;

input [7:0] a;

output reg [7:0] add;

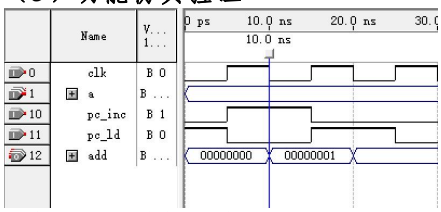
（2）功能实现

指令计数器 PC 存储当前指令在 RAM 中存放的地址。 CPU 执行一条指令，根据 PC 中存放的指令地址，将指令从 RAM 读出写入指令寄存器 IR 中，此过程称为“取指令”。在每条指令读取后，指令计数器 PC 中的地址 自动加 1，指向下一条指令在 RAM 中的存放地址。跳转指令如 JMP、JZ、JC 让程序跳转至指定地址去执行，这时 PC 需要装载跳转地址。

模型机的指令计数器 PC 是一个 8 位计数器，其的功能及封装如下：

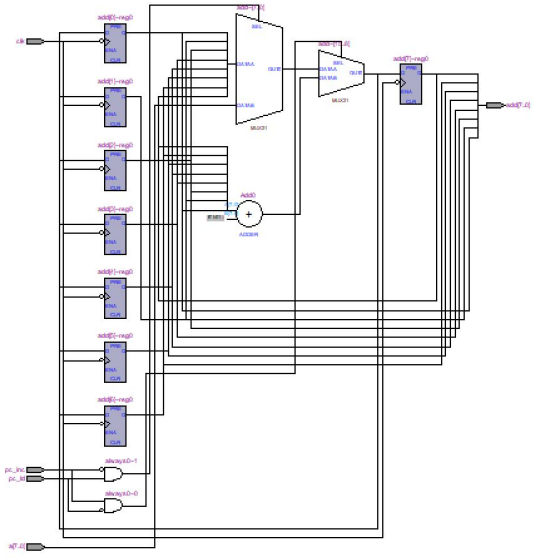
CLK	pc_inc	pc_ld	功能
⎍	1	0	add[7..0]中数据自加1
⎍	0	1	a[7..0]写入add[7..0]

（3）功能仿真验证

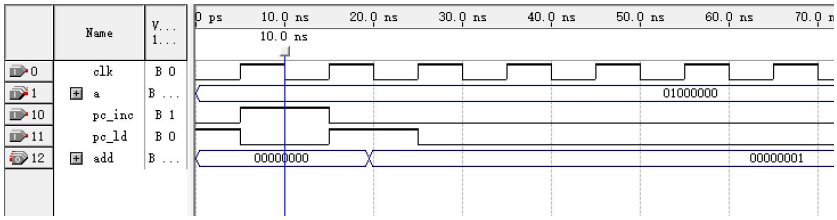


分析：对于功能仿真，5-15ns，pc_inc 为 1，pc_ld 为 0，执行地址加 1 操作，15ns-25ns,pc_inc 为 0，pc_ld 为 1，执行写入操作，将输入写入到输出中，25-40ns，pc_inc 为 0，pc_ld 为数据保持不变，正确。

（4）RTL 视图



（5）时序仿真验证



对于时序仿真，存在 9ns 左右的延迟，输出结果大致与功能仿真相同

结论：元件设计符合要求，元件存在 9ns 左右的延迟

3.2.10 寄存器组

（1）接口设计

input clk,we;

input [1:0] raa,rwba;

input [7:0] i;

output reg [7:0] s,d;

output reg [7:0] A,B,C;

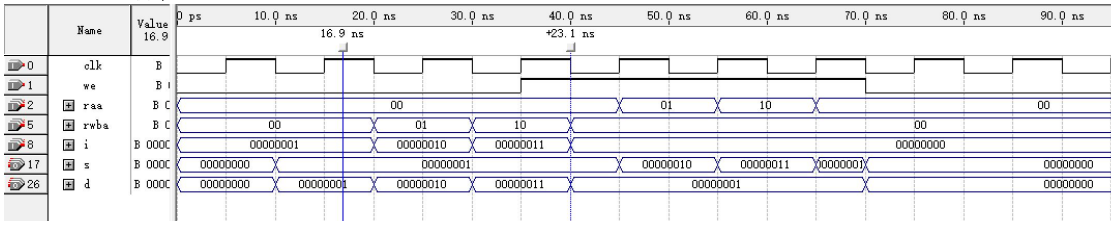
（2）功能实现

寄存器主要用来保存操作数和运算结果等信息，从而节省从 RAM 中读取操作数所需占用总线和访问存储器的时间。

模型机的通用寄存器组包含 3 个 8 位寄存器 A、B、C，可对这 3 个寄存器进行读写操作。其功能及封装如下：

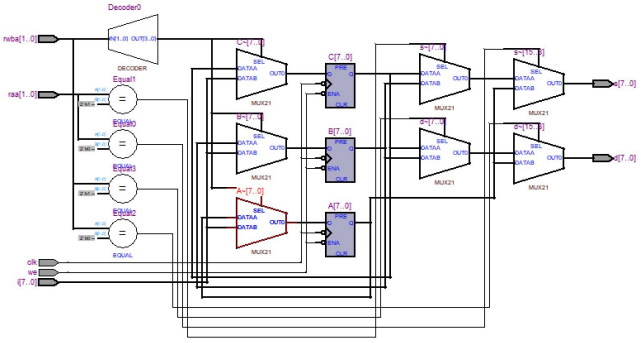
操作	CLK	WE	功能
读			根据RAA[1..0]的值从A,B,C中选择一个寄存器的值由S口输出 根据RWBA[1..0]的值从A,B,C中选择一个寄存器的值由D口输出
写	┐└	0	控制信号WE为0，根据RWBA[1..0]的值，在CLK下降沿将外部输入i写入A,B,C三个寄存器中的某个寄存器中。

（3）功能仿真验证

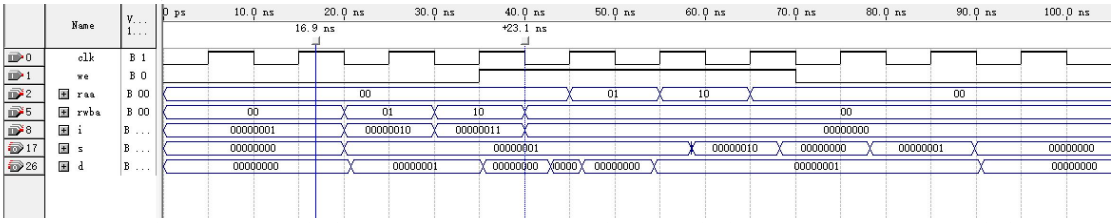


对于功能仿真，在 0-35ns，we 为 0，进行写入操作，在每个下降沿，当 rwba=00,01,10 时，将输入 i 分别写入寄存器 A,B,C 中，而在 35-65ns，we 为 1，进行读取操作，在每个下降沿，根据 raa 和 rwba 的值 s，d 输出对应寄存器的值，当 raa=00，s 输出 A 的值，raa=01，s 输出 B 的值，raa=10，s 输出 C 的值，正确

（4）RTL 视图



（5）时序仿真验证



对于时序仿真，输出 s 存在 9ns 左右的延迟，输出 d 存在 10ns 左右延迟，其余输出结果大致与功能仿真相同。

（6）时序分析

Timing Analyzer Summary

	Type	Slack	Required Time	Actual Time	From	To	From Clock	To Clock	Failed Paths
1	Worst-case tsu	N/A	None	7.015 ns	rwba[0]	B[7]	--	clk	0
2	Worst-case tco	N/A	None	12.890 ns	B[1]	d[1]	clk	--	0
3	Worst-case tpd	N/A	None	17.101 ns	rwba[0]	d[3]	--	--	0
4	Worst-case th	N/A	None	0.131 ns	i[0]	C[0]	--	clk	0
5	Total number of failed paths								0

对于时序分析，可以得到时钟输出延迟 tco 为 13.487ns，建立时间 tsu 为 7.110ns，保持时间 th 为 0.479ns，电路延迟时间 tpd 为 16.353ns

结论：元件设计符合要求，输出 s 存在 9ns 左右的延迟，输出 d 存在 10ns 左右延迟

3.2.11 RAM

（1）接口设计

输入：address[7..0]、inclock、we、outenab

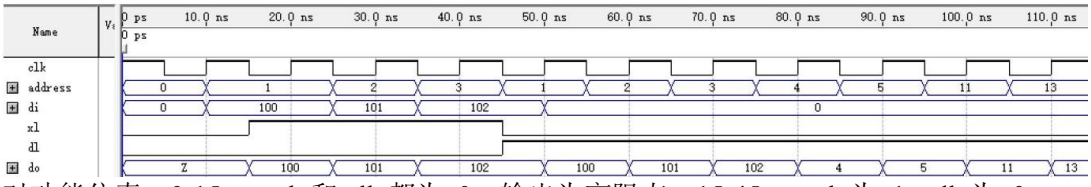
输出：dio[7..0]

（2）功能实现

半导体存储器的种类很多，从功能上可以分为只读存储器 ROM 和随机存储 RAM 两大类。随机存储器 RAM 是与 CPU 直接交换数据的内部存储器，也叫主存（内存）。它可以随时读写，而且速度很快，通常作为操作系统或其他正在运行中的程序的临时数据存储媒介。

存储元是构成存储器的存储介质，它可存储一个二进制位。由若干个存储元组成一个存储单元，然后再由许多存储单元组成一个存储器。一个存储器包含许多存储单元，每个存储单元可存放一个字节。每个存储单元的位置都有一个编号，即地址，一般用十六进制表示。一个存储器中所有存储单元可存放数据的总和称为它的存储容量。比如，一个存储器的地址码由 8 位二进制数（即 2 位十六进制数）

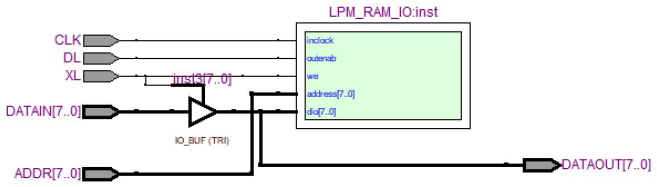
（3）功能仿真验证



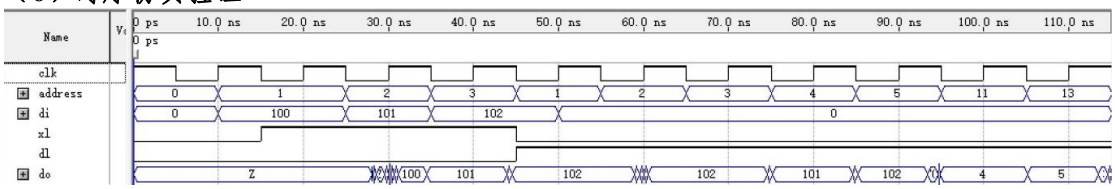
对功能仿真，0-15ns，xl 和 dl 都为 0，输出为高阻态。15-45ns，xl 为 1，dl 为 0，进行写入操作，对于三个上升沿，分别将输入信号 100,101,102 分别写入地址 1,2,3 中。45-115ns，xl 为 0，dl 为 1，进行读操作，对于每个上升沿，分别将 1,2,3,4,5,11,13 地址中的指令读出并输出，其中 4,5,11,13 地址中的指令已提前存入对应的 mif 文件中，仿真

正确。

（4）RTL 视图



（6）时序仿真验证



对于时序仿真，输出存在 11ns 左右的延迟，同时由于输入的改变导致某些位置出现冒险。

四、 系统测试

4.1 测试环境

Quartus II 9.0sp1 Web Edition
Cyclone II：EP2C5T144C8

4.2 测试代码

（使用模型机实现的指令编写一至两个程序测试模型机的正确性。）
测试代码 1:

RAM 地址	指令	指令二进制编码	指令十六制编码	
0，1：	JMP 04H	(00110000 00000100)	(30H 04H)	
4:	IN A	(00100000)	(20H)	A=10000011 (83H)
			（注：A 中的 83H 是由外部输入的）	
5:	OUT A	(01000000)	(40H)	
6:	MOV M A	(11001100)	(CCH)	C=10000000 (80H)
		（注：C 中的 80H 是在通用寄存器组设计时对 C 初始化的值）		
7:	MOV B M	(11000111)	(C7H)	B=10000011 (83H)
8:	SUB A,B	(01100001)	(61H)	A=00000000 (00H), Z=1
9,10:	JZ 10H	(00110001 00010000)	(31H 10H)	
16:	NOT A	(01010000)	(50H)	A=11111111 (FFH)
17:	AND A,C	(10110010)	(B2H)	A=10000000 (80H)
18:	ADD A,B	(10010001)	(91H)	A=00000011 (03H),C=1
19,20:	JC 18H	(00110010 00011000)	(32H 18H)	
24:	NOP	(01110000)	(70H)	
25:	RSR A	(10100000)	(A0H)	A=10000001 (81)
26:	OUT A	(01000000)	(40H)	
27:	RSL B	(10100111)	(A7H)	B=00000111 (07H)
28:	MOV A,B	(11000001)	(C1H)	A=00000111 (07H)
29:	OUT A	(01000000)	(40H)	
30:	HALT	(10000000)	(80H)	
31:	ADD A,B	(10010001)	(91H)	
32:	OUT A	(01000000)	(40H)	

测试代码 2：

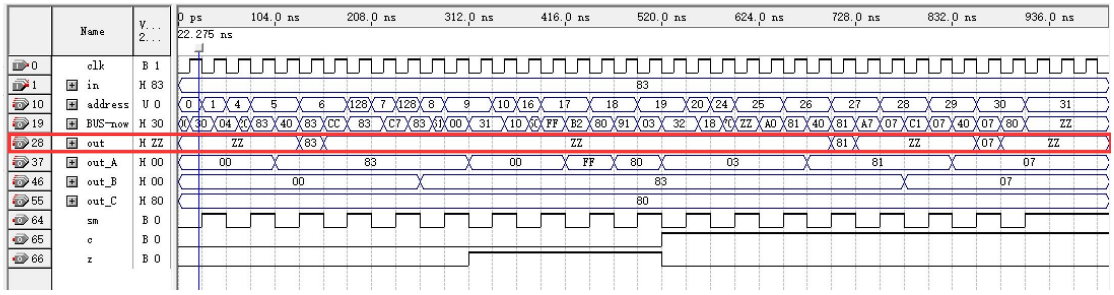
0,1：JMP 04H (00110000　00000100) (30H 04H)
4:　IN A (00100000) (20H)　　A=10000011(83H)　（注：A 中的 83H 是由外部输入的）
5:　MOV B M (11000111)(C7H)　　1）C 中存放 10000000(80H)，在通用寄存器组对 C 初始化的值
2）在 RAM 的 10000000（128）单元存放 10000011（83H）
6:　SUB A,B (01100001)(61H)　　A=00000000(00H), Z=1
7,8:　JZ 10H (00110001 00010000) (31H 10H)
9:　HALT(10000000)
16:　NOT A (01010000)(50H)　　A=11111111(FFH)
17:　AND A,C (10110010)(B2H)　　A=10000000(80H)
18:　ADD A,B (10010001)(91H)　　A=00000011(03H),C=1
19,20: JC 18H (00110010 00011000)(32H 18H)
24:　NOP (01110000) (70H)
25:　RSR A (10100000) (A0H)　　A=10000001(81H)
26:　MOV B,A (11000100) (C4H)　　B=10000001(81H)
27:　RSL B (10100111) (A7H)　　B=00000011(03H)
28　ADD B,C (10010110)(96H)　　B=10000011(83H)
29:　MOV M,B(11001101) (CDH)
30:　MOV A,M(11000011) (C3H)　　A=10000011(83H)
31:　NOT A(01010000)(50H)　　A=01111100(7CH)
32:　OUT A (01000000) (40H)　　输出为 011111100（7CH）
33:　JMP 00000000　(00110000 00000000)（30H 00H)

操作过程：C 初始化为 10000000，RAM 的 10000000 单元存放 10000011（83H）,如果外部输入 10000011，那么 JZ 和 JC 都跳转成功，执行输出 OUT A，输出为 01111100（7CH），如果改变外部输入，使其不为 10000011，那么 JZ 不成功，实现停机 HALT。

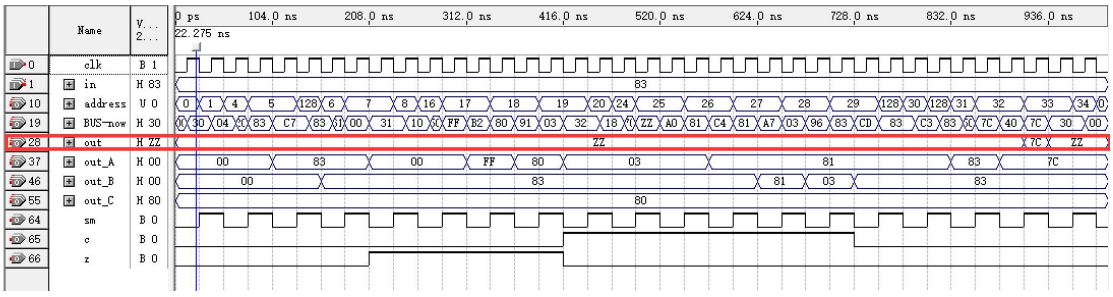
4.3 测试结果

（为了测试方便与监控变量变化，我输出了多个中间变量，红框处为输出值）

测试结果 1：



测试结果 2：



4.4 模型机性能分析

Flow Status	Successful - Fri Feb 24 22:27:52 2023
Quartus II Version	9.0 Build 184 04/29/2009 SP 1 SJ Web Edition
Revision Name	small_computer
Top-level Entity Name	small_computer
Family	Cyclone II
Device	EP2C5T144C8
Timing Models	Final
Met timing requirements	Yes
Total logic elements	203 / 4,608 (4 %)
Total combinational functions	202 / 4,608 (4 %)
Dedicated logic registers	43 / 4,608 (< 1 %)
Total registers	43
Total pins	60 / 89 (67 %)
Total virtual pins	0
Total memory bits	2,048 / 119,808 (2 %)
Embedded Multiplier 9-bit elements	0 / 26 (0 %)
Total PLLs	0 / 2 (0 %)

五、 实验总结、必得体会及建议

5.1 从需要掌握的理论、遇到的困难、解决的办法以及经验教训等方面进行总结。

（1）需要掌握的理论：基本了解了简易模型机的内部结构和工作原理。同时熟悉了指令寄存器、状态寄存器、指令计数器、寄存器的工作原理。学会使用 Verilog 语言编写电路。

（2）遇到的困难：主要是对于不同模块的整合不够熟练，由于是第一次接触到这个技能，显得很生疏。特别是如果一次搭建完整，会遇到无法确定错误位置的情况。

（3）解决方法：使用迭代增量的方法进行搭建，也就是说我先搭建其中的一部分，然后用几个输入引脚确认一下看看输出是否符合预期，这样就可以及时发现错误的位置并且作出改变。其他的问题还可以通过上网查询相关资料和询问同学后得以解决问题，并通过分析报告发现电路中的问题。有不理解的还请教了老师，不仅收获了方法还掌握了技巧。

（4）经验教训：对于电子电路的学习一定要肯动手，光是看是学不会的，一定要落到实处，多自己使用软件进行仿真，才能加深对于这门课程的理解。

5.2 对本实验内容、过程和方法的改进建议（可选项）。

对于 RTL 视图的研究十分重要，可以引导同学做到自己能看懂 RTL 视图，否则需要更改并简化写法。一定要形成能够通过 RTL 视图理顺思路、发现问题并解决问题的能力。